



MÁSTER EN INTELIGENCIA ARTIFICIAL

Detección de Outliers Mediante Autoencoders

TFM elaborado por: Alejandro Gil Villar
Tutor/a de TFM: Daniel Rubio Yagüe

- Ciudad y fecha -

Agradecimientos

En primer lugar quiero agradecer a todos aquellos que me han apoyado durante estos dos años.

Un agradecimiento especial a mi tutor Daniel , por su paciencia, dedicación, valiosas orientaciones y por creer en mis capacidades. Su guía ha sido fundamental en cada etapa de este proyecto.

Y, por supuesto, a mi familia: a mis padres y a mi hermana, por su apoyo incondicional, por motivarme día a día y por ser mi refugio y fortaleza. Vuestra ayuda ha sido fundamental.

Resumen

Este Trabajo Fin de Máster aborda la detección de fraude en transacciones con tarjeta de crédito mediante técnicas de aprendizaje automático, con especial énfasis en modelos de autoencoder para detección de anomalías. Se emplea el conocido conjunto de datos de crédito anonimizado del ULB Machine Learning Group, caracterizado por una extrema desproporción entre transacciones legítimas y fraudulentas.

La metodología incluye un preprocesado específico de las variables temporal e importe, normalización mediante *MinMaxScaler* y dos esquemas de partición: uno sin balanceo, acorde al enfoque clásico de autoencoder entrenado solo con clase normal, y otro con balanceo mediante SMOTE en el conjunto de entrenamiento. Sobre estos datos se comparan tres modelos: una línea base lineal mediante PCA, un autoencoder estándar (AE) y un autoencoder variacional (VAE), utilizando el error de reconstrucción como *score* de anomalía y umbrales fijados en el percentil 95 del MSE de entrenamiento.

Los resultados muestran que PCA ofrece un rendimiento cercano al azar (ROC-AUC $\approx$ 0,52), mientras que los autoencoders alcanzan ROC-AUC de hasta 0,95 y sensibilidades superiores al 80 % en test, manteniendo especificidades en torno al 95 %. El AE sin balanceo proporciona el mejor compromiso entre sensibilidad y especificidad, superando claramente a PCA en todos los escenarios. Se discuten las implicaciones operativas del umbral, las limitaciones derivadas del uso de variables ya transformadas por PCA y se proponen líneas de trabajo futuro basadas en filtros supervisados posteriores y en el uso de datos en crudo.

Índice general

Introducción.....	7
1.1 Aspectos Generales.....	7
1.2 Antecedentes.....	7
1.3 Objetivos.....	8
1.4 Motivación.....	8
Datos del estudio.....	9
2.1 Adquisición de fuentes de datos.....	9
2.2 Descripción de fuentes de datos.....	9
2.3 Análisis exploratorio de datos.....	10
Metodología.....	14
3.1 Preprocesado de datos.....	14
3.1.1 Transformación temporal.....	14
3.1.2 Normalización del importe (Amount).....	14
3.1.3 Reestructuración del conjunto de datos.....	15
3.2 Partición de los datos en conjuntos train y test.....	15
3.2.1 Partición sin Balanceo de Clases.....	15
3.2.2 Partición con Balanceo de Clases (SMOTE).....	16
3.3 Preparación de las Características y Etiquetas.....	17
3.4 Normalización de Datos.....	18
3.5 Funciones de Evaluación.....	19
3.5.1 Función de Curvas del Entrenamiento.....	19
3.5.2 Función Calcular Umbral.....	20
3.5.3 Función Evaluar Rendimiento Autoencoder.....	21
3.6 Callbacks.....	23
3.6.1 Early Stopping.....	23
3.6.2 Callback Personalizado para BatchHistory.....	24
3.6.3 Optimizador Adam con Decaimiento de Tasa de Aprendizaje.....	24
Modelos utilizados.....	25
4.1 PCA.....	25
4.2 Autoencoders.....	26
4.3 Autoencoder Variacional.....	27
Resultados.....	28
5.1 Sin Balanceo de Datos.....	28
5.1.1 Aplicación de modelos de aprendizaje automático.....	28
Modelo Autoencoder.....	28
Modelo Autoencoder Variacional.....	29
5.1.2 Evaluación y validación de modelos de aprendizaje automático.....	29
Modelo Autoencoder.....	30
Modelo Autoencoder Variacional.....	33
5.1.3 Comparativa de resultados.....	36
PCA.....	36
Modelo Autoencoder.....	37

Modelo Autoencoder Variacional.....	38
5.2 Con Balanceo de Datos(SMOTE).....	39
5.2.1 Aplicación de modelos de aprendizaje automático.....	39
Modelo Autoencoder.....	39
Modelo Autoencoder Variacional.....	40
5.2.2 Evaluación y validación de modelos de aprendizaje automático.....	41
Modelo Autoencoder.....	41
Modelo Autoencoder Variacional.....	44
5.2.3 Comparativa de resultados.....	47
PCA.....	47
Modelo Autoencoder.....	48
Modelo Autoencoder Variacional.....	49
Conclusiones.....	50
Referencias Bibliográficas.....	55
Apéndice.....	56
A.1 Metodología.....	56
A.1.1 Preprocesado de Datos.....	56
A.1.2 Partición sin Balanceo de Clases.....	56
A.1.3 Partición con Balanceo de Clases (SMOTE).....	57
A.1.4 Preparación de las Características y Etiquetas.....	57
A.1.5 Normalización de Datos.....	58
A.1.6 Función de Curvas del Entrenamiento.....	59
A.1.7 Función Calcular Umbral.....	60
A.1.8 Función Evaluar Rendimiento Autoencoder.....	60
A.2 Modelos.....	61
A.2.1 Autoencoder.....	61
A.2.2 Autoencoder Variacional.....	62
A.2.3 Función Sampling VAE.....	63
A.2.4 Función VAE Loss Layer VAE.....	63

Capítulo 1

Introducción.

1.1 Aspectos Generales

La detección de fraude en medios de pago constituye un problema de alto impacto en el sector financiero, tanto por las pérdidas económicas asociadas como por el deterioro de la confianza en el sistema. Este trabajo aborda dicho desafío mediante técnicas de aprendizaje automático con énfasis en autoencoders, buscando identificar transacciones anómalas dentro de un volumen grande de operaciones. El enfoque se orienta a mejorar la seguridad operativa y la experiencia del usuario, reduciendo tanto falsos negativos (fraudes no detectados) como falsos positivos (alertas indebidas).

La relevancia del problema se intensifica por la expansión del comercio electrónico y la banca digital, que incrementan la cantidad de transacciones realizadas diariamente. En este contexto, la detección temprana y precisa de patrones sospechosos se vuelve esencial para sostener la resiliencia del ecosistema financiero y preservar la confianza de clientes e instituciones.

Algunos de los desafíos que complican realizar este trabajo es la desproporción extrema de clases (entre transacciones legítimas y transacciones fraudulentas), la evolución continua de las tácticas fraudulentas y la necesidad de contener la tasa de alertas. Frente a ello, se explora un enfoque de detección de anomalías basado en reconstrucción: un autoencoder entrenado con transacciones normales debe reproducir con baja pérdida la estructura habitual y, en consecuencia, asignar errores de reconstrucción mayores a comportamientos atípicos.

1.2 Antecedentes

Tradicionalmente, la gestión del fraude ha combinado reglas de negocio (umbrales por importe, listas de bloqueo, restricciones geográficas/temporales) con modelos supervisados entrenados sobre historiales de operaciones etiquetadas. Estos enfoques resultan eficaces cuando la señal es estable y las etiquetas son abundantes, pero presentan limitaciones ante cambios de distribución, nuevos vectores de ataque y escasez de ejemplos positivos.

En paralelo, han ganado tracción las técnicas de detección de anomalías: métodos lineales (p. ej., PCA como baseline), algoritmos basados en distancias o densidades y aproximaciones de aprendizaje profundo (autoencoders y variantes), que buscan modelar la normalidad sin requerir una gran cantidad de fraudes etiquetados. En entornos con prevalencias muy bajas y comportamiento dinámico, estos métodos ofrecen una alternativa idónea o complementaria a los clasificadores supervisados convencionales.

1.3 Objetivos

El objetivo general del trabajo es diseñar y evaluar un marco de detección de fraude en transacciones financieras utilizando autoencoders como técnica principal de detección de anomalías, comparándolo con una línea base PCA y con un autoencoder variacional (VAE).

Objetivos específicos.

1. Implementar un código reproducible de preprocesado, particionado y evaluación en un dataset real de transacciones.
2. Entrenar y validar autoencoders con datos normales y estimar su umbral operativo a partir del error de reconstrucción.
3. Comparar el desempeño de PCA, AE y VAE en métricas robustas para clases raras (ROC/AUC, sensibilidad, especificidad y tasa de alertas).
4. Analizar las implicaciones operativas del umbral elegido y discutir limitaciones y mejoras potenciales.

1.4 Motivación

Una de las principales razones para realizar esta investigación mediante autoencoder es que los autoencoders destacan por su capacidad de aprender representaciones compactas, que capturan regularidades de la clase mayoritaria sin necesidad de supervisión intensiva. Mediante la reconstrucción de las entradas, proporcionan un puntaje de anomalía interpretable a nivel de sistema (pérdida de reconstrucción), útil para priorizar revisiones y ajustar políticas de alerta. Este enfoque es especialmente adecuado en escenarios altamente desbalanceados y con dinámica cambiante, donde la dependencia exclusiva de etiquetas históricas puede resultar insuficiente.

Capítulo 2

Datos del estudio.

2.1 Adquisición de fuentes de datos

El conjunto de datos se obtuvo en la plataforma Kaggle, seleccionándolo por su disponibilidad pública y por su gran cantidad de registros con más de 280.000 transacciones, lo que permite entrenar y evaluar modelos en un escenario realista de alta desproporción entre clases. Esta elección facilita, además, la reproducibilidad del estudio y la comparación con trabajos previos que emplean el mismo recurso. **Fuente:** [1]

Uno de los puntos fuertes de esta base de datos es que los datos están anonimizados, por lo que su uso es apropiado para fines académicos y de investigación, sin incluir información personal identificable.

2.2 Descripción de fuentes de datos.

El conjunto de datos analizado en este estudio proviene de transacciones con tarjetas de crédito realizadas en septiembre de 2013 por titulares europeos. Esta base de datos es fundamental para desarrollar y probar los modelos de detección de fraude, ya que incluye ejemplos tanto de operaciones legítimas como fraudulentas, permitiéndonos entrenar algoritmos que puedan identificar patrones sospechosos eficazmente.

Características del Conjunto de Datos:

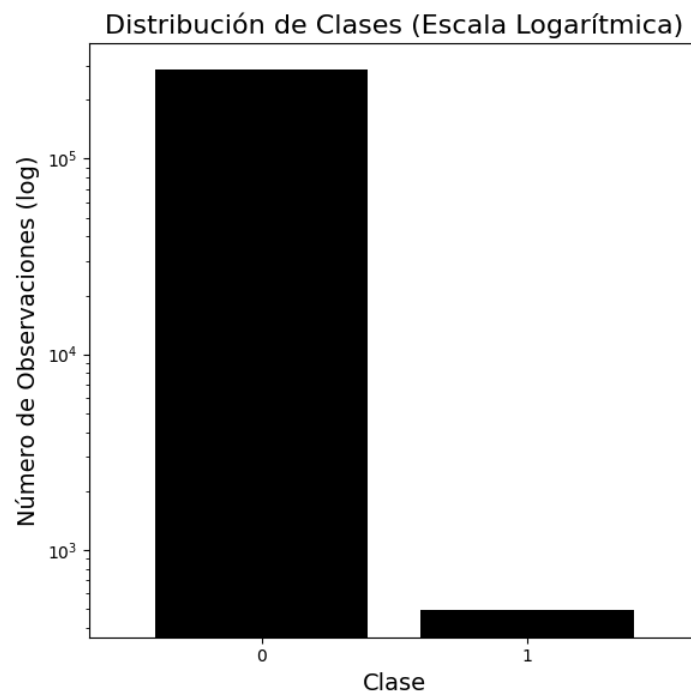
- **Tiempo (Time):** Representa el intervalo de tiempo entre cada transacción y la primera transacción en el conjunto de datos, medida en segundos. Esta variable puede ayudar a identificar patrones temporales relacionados con el fraude.
- **Variables PCA (V1-V28):** Para proteger la confidencialidad de los usuarios, las características originales han sido transformadas utilizando Análisis de Componentes Principales (PCA). Estas 28 variables resultantes son las principales componentes identificadas por PCA y no están directamente relacionadas con los atributos originales de las transacciones.
- **Cantidad (Amount):** Esta variable indica el monto de dinero involucrado en cada transacción. La distribución de montos puede variar ampliamente, desde pequeñas compras hasta grandes transferencias, y es un factor clave en la detección de actividades fraudulentas.
- **Clase (Class):** La etiqueta de cada transacción, donde 0 indica una transacción legítima y 1 señala una transacción fraudulenta. Esta variable binaria es esencial para el entrenamiento supervisado de los modelos y para evaluar su capacidad para clasificar correctamente las transacciones.

2.3 Análisis exploratorio de datos

Tras describir la procedencia, estructura y variables del conjunto de datos, se realizará un análisis exploratorio orientado a comprender tres aspectos clave para el modelado: la prevalencia de la clase positiva, la dinámica temporal de las operaciones y la distribución de los importes. Estas características descriptivas permitirán entender mejor las decisiones posteriores de preprocesado, particionado y evaluación.

Distribución de Clases:

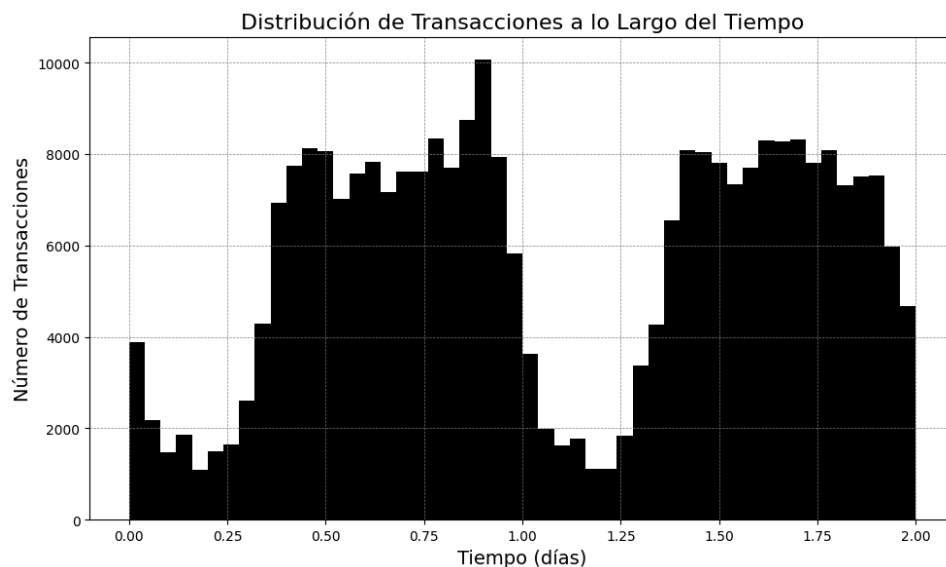
En el siguiente gráfico de barras. Se representan las dos etiquetas disponibles (0: transacción legítima; 1: transacción fraudulenta) para obtener una visión inmediata de la desproporción entre clases. El histograma evidencia que la inmensa mayoría de operaciones son normales y sólo una fracción muy reducida corresponde a fraude, lo que confirma un escenario de alto desbalance. Esta disparidad anticipa que métricas globales como la exactitud pueden resultar engañosas y subraya la conveniencia de enfoques y métricas sensibles a la clase minoritaria.



(Figura 2.1: histograma de frecuencia por clase)

Distribución de Transacciones a lo Largo del Tiempo:

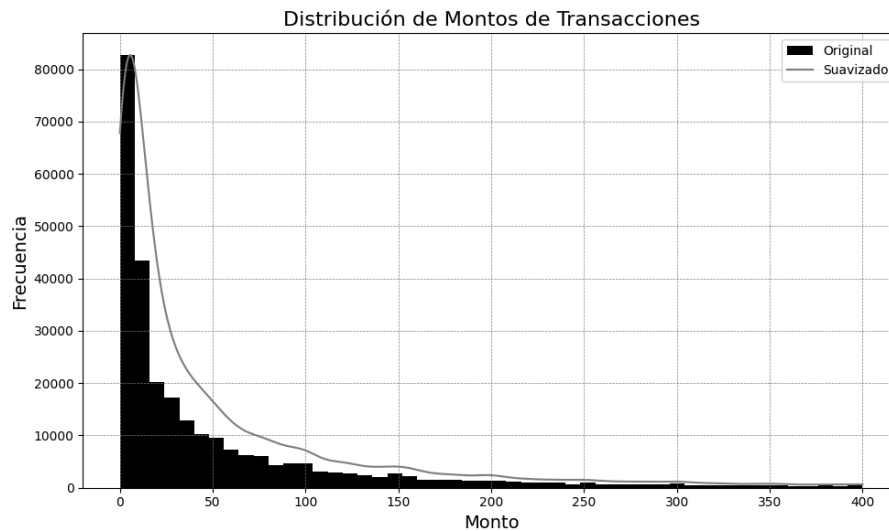
Se presenta un histograma de la variable *Time* para visualizar la intensidad de transacciones a lo largo del tiempo. El patrón observado sugiere ciclos intradiarios: menor actividad al inicio y al final del día, con mayor densidad en las horas centrales. Estos datos son coherentes con picos de uso en franjas comerciales. Esta estructura indica que el comportamiento transaccional no es uniforme y que podrían existir regularidades aprovechables por modelos sensibles a patrones secuenciales, aun cuando en este trabajo se mantenga un enfoque centrado en la reconstrucción.



(Figura 2.2: histograma de *Time* con picos de frecuencia por franja horaria)

Distribución de Montos de Transacciones:

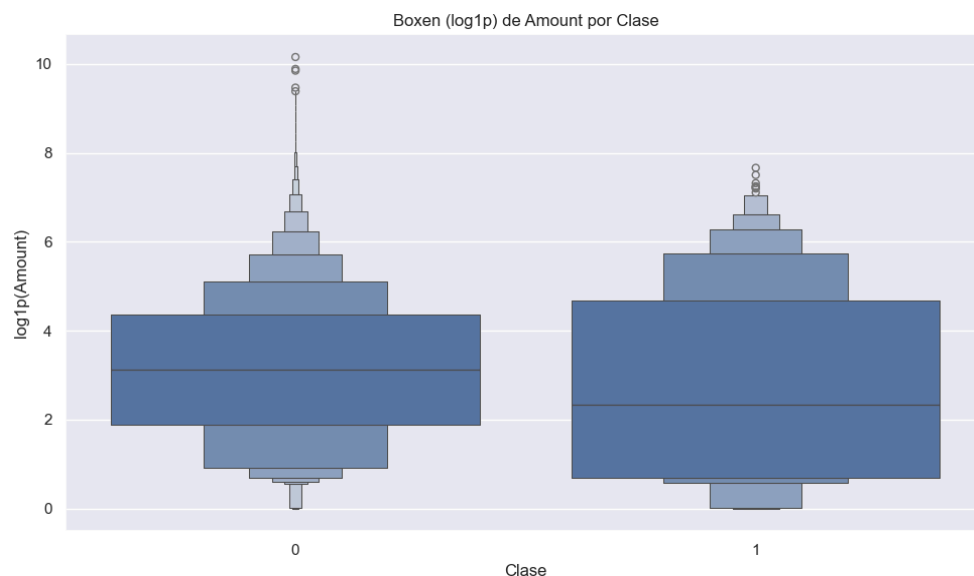
El histograma de *Amount*, complementado con una estimación de densidad (KDE), revela una distribución asimétrica con cola a la derecha: predominan importes bajos y son poco frecuentes las transacciones de grandes cantidades. Este comportamiento aconseja normalizar la variable para evitar que su escala domine la función de pérdida y, en análisis complementarios, puede considerarse una transformación logarítmica para mejorar la legibilidad de colas largas. Si bien este análisis es descriptivo y no establece causalidad, la forma de la distribución sugiere que la separación por importes no es suficiente por sí sola para discriminar el fraude, lo que refuerza la motivación de utilizar modelos capaces de capturar interacciones no lineales.



(Figura 2.3: histograma y KDE de *Amount*.)

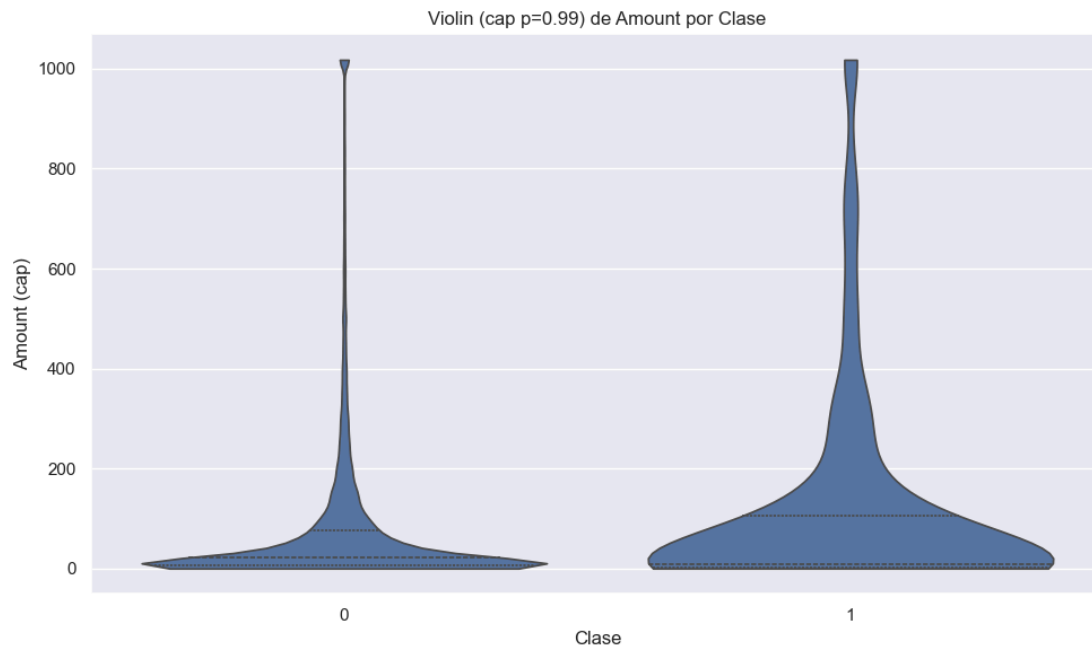
Relación entre importe y clase:

Antes de realizar la comparación entre el importe y clase , se aplica la transformación logarítmica con base 10 al Importe para comprimir valores extremos. El gráfico de cajas (boxenplot) permite contrastar medianas e intervalos entre clases sin que las colas dominen. Se observa que ambas clases (0: legítima; 1: fraudulenta) concentran la mayoría de transacciones en importes bajos, con medianas similares y dispersión comparable. El solapamiento es amplio, lo que sugiere que el importe por sí solo no discrimina de forma fiable el fraude.



(Figura 2.4: Boxen, $\log_{1p}(\text{Amount})$)

Para visualizar el cuerpo de la distribución, se recorta el Importe en el percentil 99, ocultando el 1% superior. El gráfico violín(violinplot) representa la densidad y permite apreciar la forma completa. Se observan colas largas en ambas clases y una masa principal en rangos bajos; los importes elevados son poco frecuentes y no exclusivos de la clase fraudulenta. En conjunto, ambas vistas confirman que el Importe es informativo pero insuficiente como criterio aislado y debe combinarse con otras variables o representaciones.



(Figura 2.5: Violín, Amount recortado al percentil 99)

En resumen, el análisis del capítulo 2 confirma un escenario con fuerte desbalance, patrones intradiarios y una distribución del importe claramente asimétrica, factores que condicionan de forma directa el diseño y la evaluación de los modelos de detección de fraude.

Capítulo 3

Metodología

A partir de estas evidencias, el capítulo 3 describe la metodología seguida: las etapas de preprocesado de las variables, los distintos esquemas de partición (sin balanceo y con SMOTE), los procedimientos de normalización y los controles de entrenamiento, así como el protocolo de validación definido para garantizar una correcta valoración del rendimiento de los modelos.

3.1 Preprocesado de datos.

El preprocesado persigue estabilizar las escalas, reducir sesgos y adecuar las variables a los supuestos de los modelos. Se interviene sobre las dimensiones temporal e importe, se reorganiza el *dataset* para su manejo y se generan subconjuntos por clase para análisis descriptivo. **Código:** [A.1.1]

3.1.1 Transformación temporal

Con el objetivo de capturar los distintos patrones que se producen a lo largo del día en la actividad transaccional. Se transforma la variable Time (Tiempo en segundos desde la primera transacción) aplicando la siguiente fórmula $Hora\ del\ dia = \left(\frac{Time(s)}{3600} \right)$, lo que devuelve la hora del día (0–24) asociada a cada transacción. Esta transformación del campo revela patrones cíclicos (p. ej., menor actividad nocturna y mayor en horas comerciales) potencialmente relevantes para la detección de anomalías.

3.1.2 Normalización del importe (*Amount*)

Para evitar problemas con valores nulos en la columna del Importe de las transacciones, se suma un desplazamiento mínimo (0,00001) y se aplica una transformación logarítmica. Esta transformación comprime los valores extremos y estabiliza la varianza, favoreciendo que la función de pérdida no quede dominada por importes muy elevados.

Dado que la distribución del importe presenta sesgo positivo marcado, la transformación logarítmica mejora la legibilidad estadística y la eficiencia del aprendizaje sin alterar el orden relativo de los importes..

La elección de transformar el Importe se apoya en el EDA (Sección 2.3), donde se observó una asimetría marcada y colas largas debido al gran número de operaciones pequeñas y pocas operaciones grandes que podrían distorsionar el ajuste si no se estabiliza la escala.

3.1.3 Reestructuración del conjunto de datos

Dado que se han creado nuevas columnas y para realizar tanto el particionado de los datos como el entrenamiento de los modelos, se necesita una estructura clara del conjunto de datos. Se va a reestructurar el conjunto de datos de la siguiente forma:

1. **Reorganización de Columnas:** Para una mejor legibilidad y manejo del conjunto de datos, `log10_amount` y `Class` se mueven al final del `DataFrame`.
2. **Eliminación de Características Redundantes:** Tras generar `log10_amount`, la columna `Amount` se elimina, ya que hay dos columnas con la misma información por lo que para evitar colinealidad y mantener una única representación estable del importe se mantiene solamente la columna de `log10_amount`.

Con las variables estabilizadas y el *dataset* organizado, el siguiente apartado describe el esquema de partición en conjuntos de entrenamiento y prueba, tanto en el escenario sin balanceo como con SMOTE.

3.2 Partición de los datos en conjuntos train y test.

El particionado busca garantizar reproducibilidad, representatividad y ausencia de fuga de información. Para ello se emplean semillas fijas(10) y cuando procede, estratificación por clase para preservar la prevalencia.

Se comparan dos esquemas: (i) sin balanceo, acorde al enfoque de detección de anomalías con autoencoder entrenado solo en clase normal; y (ii) con balanceo (SMOTE), orientado a un entrenamiento supervisado con clases más equilibradas.

3.2.1 Partición sin Balanceo de Clases

Se ajusta el autoencoder únicamente con transacciones legítimas y evalúa su capacidad de detectar anomalías en un entorno con prevalencia real. **Código:** [\[A.1.2\]](#)

Procedimiento.

1. **Mezcla de Datos No Fraudulentos:** Se mezclan aleatoriamente las transacciones no fraudulentas para evitar sesgos debidos al orden de los registros (semilla fija).
2. **Creación de Conjuntos de Entrenamiento y Validación:** Con `train_test_split` se divide la clase 0 en entrenamiento y validación, de modo que el autoencoder aprende un patrón de reconstrucción propio de la normalidad y se controla el sobreajuste en validación.

3. **Reserva de fraudes para validación y test.** Se separa un 10% de la clase 1 (fraude) para validación, manteniéndola fuera del entrenamiento; el resto de fraudes pasa al conjunto de test. Reservando el 10% de la clase fraudulenta para validación asegura suficientes positivos para una estimación estable de sensibilidad sin empobrecer el conjunto de test, que mantiene la mayor parte de los casos de fraude para la evaluación final.

4. **Combinación de Conjuntos de Datos:**

- **Validación:** combinación de normales (procedentes del split de la clase 0) y del **10%** de fraudes reservados.
- **Test:** combinación del remanente de normales y del 90% restante de fraudes. Este diseño aproxima un flujo operativo, permitiendo estimar métricas con la prevalencia real o próxima a la real.

Todas las transformaciones (p. ej., escalado) se ajustan exclusivamente con el conjunto de entrenamiento (clase 0) y se aplican después a validación y test, evitando así fuga de información.

3.2.2 Partición con Balanceo de Clases (SMOTE)

Se construye un conjunto de entrenamiento balanceado que contiene registros normales y fraudulentos, aumentando la representación de la clase minoritaria para facilitar el aprendizaje supervisado y la comparación con el escenario no balanceado. **Código:** [A.1.3]

Procedimiento.

1. **Preparación de los Datos Normales y Fraudulentos**

- **Normalización de las transacciones normales:** se mezclan aleatoriamente las transacciones no fraudulentas para evitar cualquier sesgo en el orden de los registros.
- **División del dataset:** se utilizan particiones `train_test_split` para crear conjuntos de entrenamiento, validación y prueba tanto para transacciones normales como para las fraudulentas. De esta manera, cada subconjunto contiene ejemplos de ambas clases.

2. **Construcción del Conjunto de Entrenamiento Combinado**

- Se unen las transacciones normales y fraudulentas correspondientes al entrenamiento.
- Se crea un vector de etiquetas asociado, donde 0 indica transacciones normales y 1 indica transacciones fraudulentas.
- En este punto, el conjunto de entrenamiento refleja la desproporción natural entre las clases, siendo necesario aplicar técnicas de balanceo.

3. **Aplicación de SMOTE para Balanceo**

Se emplea la técnica de sobremuestreo sintético SMOTE para generar nuevas instancias de la clase fraudulenta a partir de sus vecinos más próximos, metodología de referencia en el tratamiento de clases desbalanceadas (Chawla, Bowyer, Hall, & Kegelmeyer, 2002). La implementación se realiza mediante la librería *imbalanced-learn*, que integra SMOTE dentro del ecosistema de *scikit-learn* y facilita su uso en pipelines de preprocesado y modelado (Lemaître, Nogueira, & Aridas, 2017).

- Se aplica exclusivamente sobre *X_train* y *y_train* para evitar fugas hacia validación/test.
- La clase mayoritaria (transacciones normales) se mantiene intacta.
- La clase minoritaria (fraudes) se incrementa artificialmente mediante la generación de nuevos ejemplos sintéticos hasta alcanzar el número necesario de instancias definido por la proporción deseada.
- El resultado es un conjunto de entrenamiento equilibrado que facilita el aprendizaje del modelo, reduciendo el riesgo de sesgo hacia la clase mayoritaria.

4. Preparación de Conjuntos de Validación y Test

- Finalmente, se integran transacciones fraudulentas reales en los conjuntos de validación y prueba.
- Estos se mezclan de forma aleatoria para simular un entorno realista en el que el modelo debe enfrentarse a un flujo heterogéneo de transacciones normales y fraudulentas.

Definidos los esquemas de partición, el siguiente apartado (3.3) detalla la preparación de características y etiquetas, así como las garantías adicionales para aplicar transformaciones de forma coherente con el diseño experimental.

3.3 Preparación de las Características y Etiquetas

Extracción de etiquetas y definición de matrices de entrada

1. **Etiquetas (y).** Se separa la columna *Class* como variable objetivo binaria (0: legítima; 1: fraudulenta) en cada subconjunto correspondiente (*train*, validación y *test*).
2. **Características (X).** Se eliminan de las matrices de entrada todas las columnas de etiqueta, conservando únicamente las variables predictoras (p. ej., *Time* transformado, *log10_amount*, y *V1–V28*). Para poder entrenar los modelos únicamente con datos de entrada

Código: [A.1.4]

3.4 Normalización de Datos

Para optimizar el rendimiento de los modelos de aprendizaje automático en el análisis de fraude, aplicamos un proceso de normalización utilizando MinMaxScaler. Este proceso involucra los siguientes pasos clave. **Código:** [\[A.1.5\]](#)

La normalización se implementa mediante la clase MinMaxScaler de la librería scikit-learn, que proporciona herramientas estandarizadas para el preprocesado y la construcción de modelos en Python, manteniendo una interfaz coherente a lo largo de todo el pipeline (Pedregosa et al., 2011).

1. **Inicialización de MinMaxScaler:**

Inicializamos MinMaxScaler para escalar las características dentro de un rango predeterminado, generalmente entre 0 y 1. Este paso prepara al escalador para ajustarse a los datos de entrenamiento y asegura una transformación uniforme de todos los conjuntos de datos.

2. **Ajuste de MinMaxScaler con Datos de Entrenamiento:**

El escalador se ajusta con el conjunto de entrenamiento para determinar los valores mínimos y máximos de cada característica. Este ajuste permite establecer los parámetros necesarios para la normalización, asegurando transformaciones coherentes en todos los datos.

3. **Transformación de Conjuntos de Entrenamiento y Validación:**

Los conjuntos de entrenamiento y validación se transforman utilizando los parámetros ajustados. Esta normalización facilita el aprendizaje del modelo y mejora la precisión, al hacer que las características de diferentes magnitudes sean directamente comparables.

4. **Transformación del Conjunto de Prueba:**

El conjunto de prueba se transforma utilizando los mismos parámetros de escala ajustados previamente. Esto garantiza que se normalice de manera idéntica a los conjuntos de entrenamiento y validación, permitiendo evaluaciones precisas del rendimiento del modelo en datos no vistos.

Con los datos correctamente normalizados, el siguiente apartado (3.5) presenta las funciones de evaluación y métricas empleadas, así como los criterios de selección de umbrales para la interpretación operativa de los resultados.

3.5 Funciones de Evaluación

En esta sección se describen las funciones principales empleadas para evaluar los modelos, tanto durante el entrenamiento como en los conjuntos de validación y test.

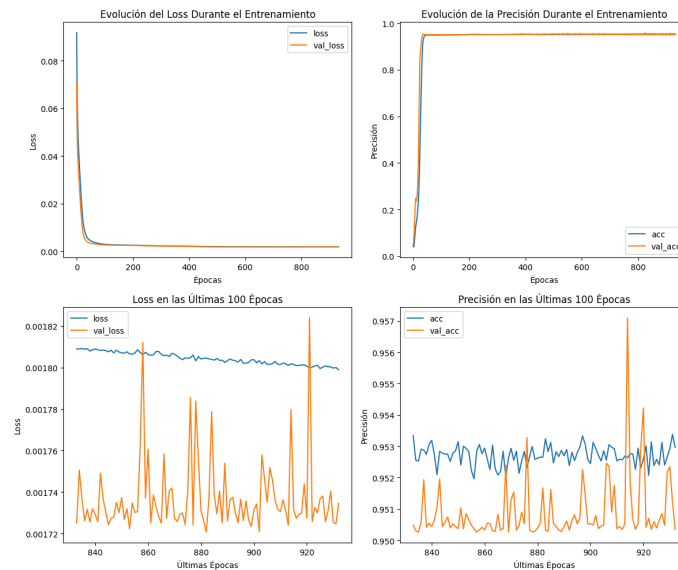
Estas herramientas permiten, conjuntamente, analizar la convergencia del modelo, diagnosticar posibles problemas de sobreajuste o subajuste y cuantificar su capacidad para distinguir entre transacciones normales y fraudulentas.

3.5.1 Función de Curvas del Entrenamiento

Esta función está diseñada para proporcionar una visión exhaustiva del proceso de entrenamiento de un modelo, visualizando la evolución de tres métricas claves: el loss (pérdida), la acc (precisión) y el mse (error cuadrático medio). Estas métricas se presentan tanto para el conjunto de entrenamiento como para el de validación. **Código:** [\[A.1.6\]](#)

Pasos:

1. **Evolución del Loss:** El primer gráfico muestra cómo el loss y el val_loss cambian a lo largo de todas las épocas. Esta visualización es fundamental para entender cómo el modelo está aprendiendo y si está sufriendo de sobreajuste o subajuste.
2. **Evolución de la Precisión:** El segundo gráfico muestra la trayectoria de la acc y val_acc a través de las épocas, proporcionando una visión de cómo la capacidad del modelo para clasificar correctamente los datos mejora con el entrenamiento.
3. **Foco en las Últimas 100 Épocas para el Loss:** El tercer gráfico se centra en las últimas 100 épocas para el loss y val_loss, proporcionando una vista detallada de la convergencia del modelo hacia el final del entrenamiento.
4. **Foco en las Últimas 100 Épocas para la Precisión:** De manera similar, el cuarto gráfico enfoca en la precisión (acc y val_acc) durante las últimas 100 épocas, permitiendo una inspección más cercana de las mejoras o cambios finales en la capacidad predictiva del modelo.
5. **Visualización y Organización:** Cada gráfico incorpora leyendas para diferenciar entre los datos de entrenamiento y validación. La disposición de los subgráficos se ajusta para evitar superposiciones y garantizar una interpretación clara de los datos.



(Figura 3.1: Resultado de la Función de Curvas de Entrenamiento)

3.5.2 Función Calcular Umbral

Esta función tiene como proposito definir un punto de corte para clasificar como *outlier* aquellas observaciones cuya reconstrucción sea inusualmente mala respecto al patrón de normalidad aprendido por el autoencoder. **Código:** [A.1.7]

Pasos.

1. **Reconstrucción en entrenamiento.** Se obtiene, para cada muestra del conjunto de características, su versión reconstruida por el autoencoder.
2. **Error de reconstrucción.** Para cada muestra se calcula el MSE de reconstrucción comparando la entrada con su reconstrucción (promedio del cuadrado de las diferencias componente a componente). Este valor resume “cuánto se aleja” cada punto del patrón de normalidad.
Se interpreta el MSE como puntuación de anomalía: a mayor MSE, mayor probabilidad de *outlier*.
3. **Selección del umbral.** Se fija el umbral τ en el percentil 95 de la distribución de MSE obtenida en el paso anterior. Todas las observaciones con $MSE \geq \tau$ se consideran outliers (5% superior), asumiendo que reconstrucciones muy pobres reflejan desviaciones de la norma.
El percentil 95 ofrece un criterio simple que limita la tasa de alertas al ~5% del entrenamiento, favoreciendo recall sin inflar en exceso los falsos positivos en la fase inicial de modelado.

La Salida de la función es el valor del umbral el cual se usará el funcion para evaluar el rendimiento del Autoencoder

3.5.3 Función Evaluar Rendimiento Autoencoder

Esta función mide la capacidad del autoencoder para distinguir transacciones normales de anómalas en validación, usando el MSE como puntuación continua y el umbral τ para clasificación binaria.

Dado el marcado desbalance entre clases, la evaluación se apoya en métricas específicas como la curva ROC y el área bajo la curva (AUC), que permiten analizar la capacidad discriminativa de los modelos de forma independiente del umbral de decisión (Fawcett, 2006). **Código:** [\[A.1.8\]](#)

Pasos.

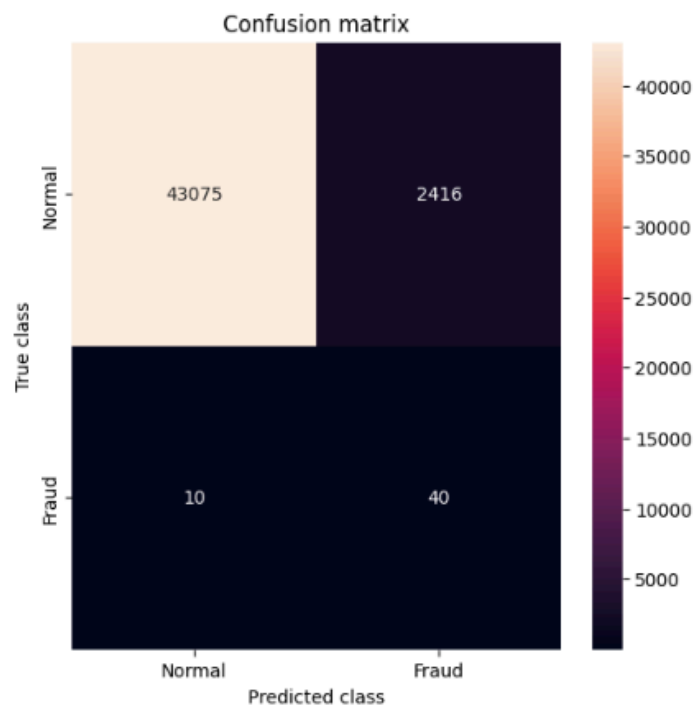
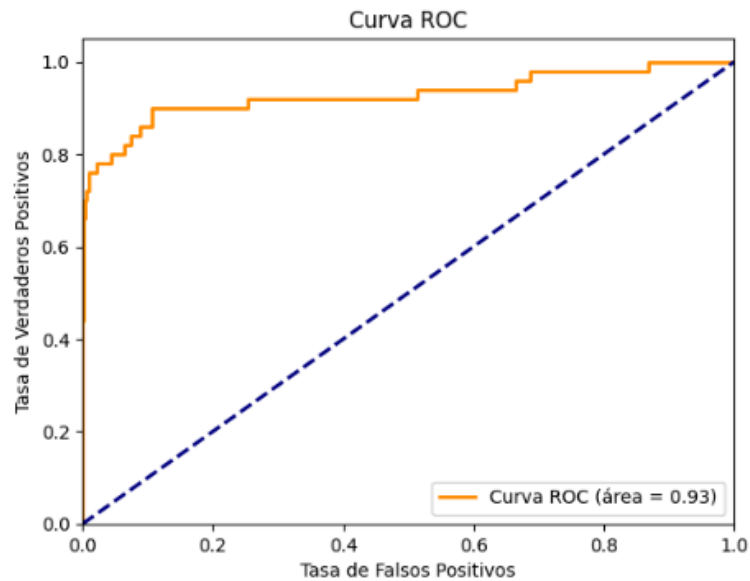
1. **Cálculo del Umbral:** Se obtiene el umbral para la detección de outliers a partir de la función `calcular_umbral`.
2. **Reconstrucción y MSE en validación.** Se reconstruyen las muestras de validación y se calcula su MSE (puntuación de anomalía).
3. **MSE medio.** Se informa el error de reconstrucción MSE medio en el conjunto de validación, proporcionando una medida de la calidad de las reconstrucciones del autoencoder en datos no vistos durante el entrenamiento.
4. **Curva ROC y AUC.** Con el MSE como *score* (a mayor MSE, mayor sospecha), se traza la ROC y se calcula el AUC, que resume la separabilidad global para todos los umbrales posibles.
5. **Clasificación por umbral τ y matriz de confusión.** Se etiqueta como outlier si $MSE \geq \tau$ y como normal en caso contrario, generando predicciones que se comparan con las etiquetas reales para producir la matriz de confusión y luego se calculan métricas claves como:
 - **Sensibilidad (Recall/TPR):** fraudes correctamente detectados.
 - **Especificidad (TNR):** normales correctamente identificadas.
 - **Precisión (PPV):** proporción de positivos predichos que realmente son fraude.

El resultado de esta función es el valor del umbral, Error de reconstrucción MSE Medio, Curva ROC y Matriz de Confusión junto sus métricas.

Umbral para la detección de outliers: 0.01704364512831514

1424/1424 [=====] - 3s 2ms/step

Error de reconstrucción MSE medio en el conjunto de validación: 0.010748038791467388



% de transacciones etiquetadas como fraude que fueron correctas (precisión): $40/(2416+40) = 1.63\%$
% de transacciones fraudulentas que fueron detectadas con éxito (sensibilidad): $40/(10+40) = 80.00\%$
% de transacciones normales que fueron identificadas correctamente (especificidad): $43075/(43075+2416) = 94.69\%$

(Figura 3.2: Resultado de la Función de Evaluación)

Establecidos el umbral y las métricas, el siguiente apartado (3.6) presenta los callbacks y controles de entrenamiento empleados para favorecer la generalización y monitorizar la convergencia del modelo.

3.6 Callbacks

Para asegurar un entrenamiento estable y una evaluación rigurosa del autoencoder, se incorporan mecanismos de monitorización durante el ajuste del modelo. Estos *callbacks* permiten detectar a tiempo el sobreajuste, registrar el comportamiento intra-época de las métricas y ajustar de forma dinámica la tasa de aprendizaje para favorecer la convergencia. A continuación se describen los componentes utilizados *EarlyStopping*, un callback personalizado de historial por *batch* y Adam con decaimiento exponencial y su función específica en el pipeline

3.6.1 Early Stopping

El callback *EarlyStopping* detiene el entrenamiento cuando la métrica monitorizada deja de mejorar, mitigando el sobreajuste y reduciendo el coste computacional. En este trabajo se monitoriza *val_loss* (pérdida de validación) con los siguientes parámetros:

- **min_delta = 1e-6**: mejora mínima considerada como progreso real.
- **patience = 50**: número máximo de épocas consecutivas sin mejora antes de detener. Este valor permite amortiguar la variabilidad estocástica del *loss* y consolidar mejoras graduales propias del entrenamiento de autoencoders, evitando detenciones prematuras.
- **restore_best_weights = True**: al finalizar, se restauran automáticamente los pesos correspondientes al mejor valor de *val_loss* observado, asegurando que las evaluaciones posteriores se realicen sobre el mejor modelo alcanzado durante el entrenamiento.

En autoencoders, *val_loss* (típicamente MSE/MAE de reconstrucción) es un indicador directo de la fidelidad de reconstrucción sobre datos no vistos; por ello, es la métrica usada para activar la parada temprana.

3.6.2 Callback Personalizado para BatchHistory

BatchHistory es un callback definido para **registrar métricas por batch** durante el entrenamiento. Su funcionamiento es:

- **Inicialización:** al comienzo del entrenamiento crea listas internas para almacenar loss y acc por batch.
- **Registro:** después de que finalice cada batch almacena el valor de pérdida y, si está disponible, el de exactitud reportado por Keras.
- **Control de flujo:** al término de la primera época, fuerza la interrupción del entrenamiento.

3.6.3 Optimizador Adam con Decaimiento de Tasa de Aprendizaje

Se emplea Adam como optimizador, combinándolo con un programador de tasa de aprendizaje (learning rate scheduler) de tipo ExponentialDecay. Esta configuración proporciona:

- Adaptación de gradientes (Adam), mejorando la convergencia en espacios de alta dimensión.
- Disminución progresiva de la tasa de aprendizaje, lo que favorece pasos más finos a medida que el modelo se aproxima a un mínimo.

Parámetros típicos del scheduler (según la implementación utilizada):

- **initial_learning_rate:** tasa de aprendizaje inicial.
- **decay_steps** y **decay_rate:** controlan la velocidad del decaimiento exponencial.
- **staircase (opcional):** si es True, el decaimiento se aplica en saltos discretos; si es False, es suave/continuo.

Relación con EarlyStopping. El decaimiento progresivo facilita la estabilización de la pérdida de validación, y EarlyStopping actúa como mecanismo de seguridad cuando ya no se observan mejoras relevantes, devolviendo los mejores pesos.

Con los mecanismos de entrenamiento y monitorización definidos, el capítulo 4 presenta los modelos utilizados (PCA, Autoencoder y Autoencoder Variacional) y sus configuraciones específicas para la comparación experimental.

Capítulo 4

Modelos utilizados

Este capítulo describe los modelos considerados, su fundamento y la configuración aplicada en el estudio. En primer lugar se presenta PCA como línea base lineal y de bajo coste para reducción de dimensionalidad y obtención de puntuaciones de anomalía. A continuación, se detalla el Autoencoder (AE) como enfoque no lineal de reconstrucción entrenado sobre la clase normal, y, por último, el Autoencoder Variacional (VAE), que incorpora una estructura probabilística en el espacio latente. En cada caso se justifica su conveniencia para la detección de fraude en datos desbalanceados y se explican los criterios de uso que se emplearán en la comparación posterior.

4.1 PCA

PCA es una técnica lineal de reducción de dimensionalidad que proyecta los datos sobre un subespacio ortogonal de menor dimensión maximizando la varianza explicada (Jolliffe, 2002). La proyección se obtiene a partir de los autovectores de la matriz de covarianza y permite representar la información relevante en un conjunto compacto de componentes principales. Para detección de anomalías puede emplearse: la distancia en el espacio reducido o el error de reconstrucción al invertir la proyección, asumiendo que las observaciones “normales” se reproducen mejor con pocas componentes que las anómalas.

Configuración empleada

Se ajusta un PCA sobre los datos de entrenamiento para obtener una representación de baja dimensión (p. ej., 30 componentes) que conserve la mayor parte de la varianza. La reconstrucción y las distancias en el subespacio se utilizan como puntuación de anomalía para trazar curvas ROC/PR y comparar con los otros enfoques.

Por qué utilizarlo en este estudio

- **Línea base interpretable:** establece un mínimo de desempeño con un método clásico, rápido y reproducible.
- **Contraste lineal vs. no lineal:** al ser lineal, sirve para valorar cuánto aportan los modelos no lineales (Autoencoder, VAE) sobre las V1–V 28 (ya derivadas de un PCA de anonimización).
- **Coste computacional bajo:** útil en escenarios de referencia o como filtro preliminar de outliers.

4.2 Autoencoders.

Un autoencoder es una red neuronal que aprende a reconstruir su entrada a través de una representación latente de menor dimensión. Consta de un *encoder* (comprime) y un *decoder* (reconstruye); el entrenamiento se basa en minimizar una pérdida de reconstrucción, normalmente sobre datos considerados “normales” (Goodfellow et al., 2016). El AE aprende así un “patrón de normalidad”: las muestras coherentes con ese patrón se reconstruyen con bajo error, mientras que las atípicas producen errores de reconstrucción elevados. Este error (p. ej., MSE) se usa como puntuación de anomalía.

Configuración empleada

- Entrenamiento no supervisado con transacciones legítimas; validación y prueba incluyen ambas clases para el conjunto sin balanceo de datos.
Para el conjunto con SMOTE se realiza un entrenamiento no supervisado con transacciones de ambos tipos, además la validación y prueba también tendrán datos de ambas clases.
- Pérdida de reconstrucción (MSE/MAE) y parada temprana para evitar sobreajuste.
- Umbral fijado en validación, mediante el percentil 95 del MSE estimado en entrenamiento para clasificar anómalos.

Por qué utilizarlo en este estudio

- Capta relaciones no lineales entre variables (incluidas V1–V28), superando el marco lineal del PCA.
- Adecuado para desbalance extremo: Para conseguir un buen resultado no necesita una gran cantidad de fraudes etiquetados ya que aprende la estructura de lo normal.
- Robustez ante deriva moderada: al centrarse en reconstrucción de patrones generales, puede adaptarse mejor a cambios graduales que reglas rígidas.

Limitaciones: elección del umbral y sensibilidad a la escala de las variables; interpretabilidad menor que PCA (la latente no es directamente semántica).

Código: [\[A.2.1\]](#)

4.3 Autoencoder Variacional.

El VAE es un modelo generativo que introduce una formulación probabilística del espacio latente. El encoder no produce un único punto, sino los parámetros (media y varianza) de una distribución normal desde la cual se muestrean representaciones latentes; el decoder reconstruye la entrada a partir de dichas muestras. El entrenamiento optimiza una cota inferior de la evidencia (ELBO) que combina una pérdida de reconstrucción con un término de divergencia KL para regularizar la latente (Kingma & Welling, 2014). Para detección de anomalías se emplea como puntuación, de nuevo, el error de reconstrucción.

Configuración empleada

- Función objetivo que combina pérdida de reconstrucción y regularización KL para obtener latentes estructuradas y mejor generalización.
- Umbralización análoga al AE mediante el percentil 95 para clasificar anomalías.

Por qué utilizarlo en este estudio

- **Marco probabilístico:** la regularización latente impone una estructura que puede mejorar la generalización y la estabilidad frente a ruido.
- **Score flexible:** permite usar tanto reconstrucción como desviación en latente (KL) para puntuar rareza.
- **Comparación informativa:** contrasta con AE si la densidad latente añade valor en un dataset con variables ya transformadas (V1–V28).

Limitaciones: puede sub-reconstruir detalles finos por la regularización (efecto de “suavizado”), y la calibración del score (reconstrucción) requiere validación cuidadosa.

Código: [\[A.2.2\]](#), [\[A.2.3\]](#), [\[A.2.4\]](#)

Definidos los modelos y sus configuraciones, el siguiente capítulo describe los resultados obtenidos en los dos escenarios de evaluación sin balanceo y con SMOTE, utilizando como puntuación el error de reconstrucción y el umbral τ fijado en entrenamiento. Se presentan curvas ROC/AUC, la matriz de confusión y métricas derivadas (sensibilidad, especificidad y precisión) en validación y test. Esta evidencia permitirá comparar, de forma consistente, el desempeño de PCA, AE y VAE en la detección de transacciones anómalas.

Capítulo 5

Resultados

En este capítulo se presentarán los distintos modelos tanto para el conjunto Sin Balanceo de Datos como para el conjunto con SMOTE, seguidamente se analizarán los entrenamientos de los distintos modelos como sus resultados con el conjunto de validación para entender porque se han elegido estos 4 modelos de datos. Y finalmente se compara y analiza el resultado de los 4 modelos con los conjuntos de test.

Los modelos de autoencoder y autoencoder variacional se implementan utilizando Keras sobre TensorFlow, lo que permite definir arquitecturas profundas de forma modular e integrar de manera directa callbacks de entrenamiento como *EarlyStopping* o programadores de tasa de aprendizaje dentro del mismo entorno de experimentación (Chollet, 2015).

5.1 Sin Balanceo de Datos

5.1.1 Aplicación de modelos de aprendizaje automático.

Modelo Autoencoder

El autoencoder se plantea como un mecanismo de reconstrucción de la normalidad a partir de una arquitectura secuencial con dos etapas. En la codificación, una primera capa densa recibe la entrada de dimensión *input_dim* y aplica ELU para preservar no linealidades; a continuación, una normalización por *batch* estabiliza las activaciones y acelera el ajuste, y un dropout del 10% introduce aleatoriedad controlada para mitigar el sobreajuste. La compresión progresa con tres capas densas intermedias de 32, 16 y 8 unidades (todas con ELU), hasta llegar a la capa latente de 4 unidades también con ELU, sobre la que se aplica regularización L1 para promover representaciones parcimoniosas y mejor generalización.

La decodificación invierte simétricamente el camino: capas densas de 8, 16 y 32 neuronas (activación ELU) reconstruyen gradualmente la estructura de entrada, de nuevo con normalización por *batch* y dropout del 10% para estabilizar el proceso. La capa final es densa con número de unidades igual a *input_dim* y activación sigmoide, encargada de producir la reconstrucción y, con ello, el error de reconstrucción que después se utilizará como puntuación de anomalía. El ajuste se realiza con Adam (tasa de aprendizaje 0.000005), elegido por su adaptación por parámetro en paisajes de error complejos y por facilitar una convergencia suave que reduce el riesgo de sobreajuste.

Modelo Autoencoder Variacional

El VAE introduce una formulación probabilística del espacio latente. El codificador recibe tensores de entrada con 30 características y, a través de capas densas con ELU, estima los parámetros de una distribución latente: z_mean y z_log_var . Mediante la reparametrización se obtiene z , una muestra de dicha distribución que actúa como representación comprimida. El decodificador toma z y, con otra secuencia de capas densas con ELU, reconstruye una salida con la misma forma que la entrada original. Para estabilizar y regularizar, el modelo incorpora normalización por *batch* en puntos críticos del *encoder* y del *decoder*, así como dropout tras capas densas seleccionadas.

La función de pérdida combina dos términos: la pérdida de reconstrucción (diferencia cuadrática media entre entrada y salida) que asegura fidelidad, y la divergencia KL que alinea la distribución latente aprendida con un prior (normal) y estructura el espacio latente para favorecer la generalización. El optimizador es Adam con una tasa de aprendizaje de 0.00001, suficiente para avanzar con pasos controlados en presencia del doble objetivo (reconstrucción + KL) y la consiguiente sensibilidad del entrenamiento a las escalas de activación.

5.1.2 Evaluación y validación de modelos de aprendizaje automático.

En el escenario sin balanceo de datos se evalúa la capacidad de los modelos de reconstrucción para distinguir comportamientos habituales de transacciones anómalas manteniendo la prevalencia real. Ambos modelos comparten el mismo protocolo: datos normalizados con parámetros ajustados en entrenamiento, monitorización con *early stopping* y decisión operativa basada en un umbral derivado del percentil 95 del error de reconstrucción en *train*. A continuación se describen el proceso de entrenamiento y los resultados en validación para el Autoencoder (AE) y el Autoencoder Variacional (VAE).

Modelo Autoencoder

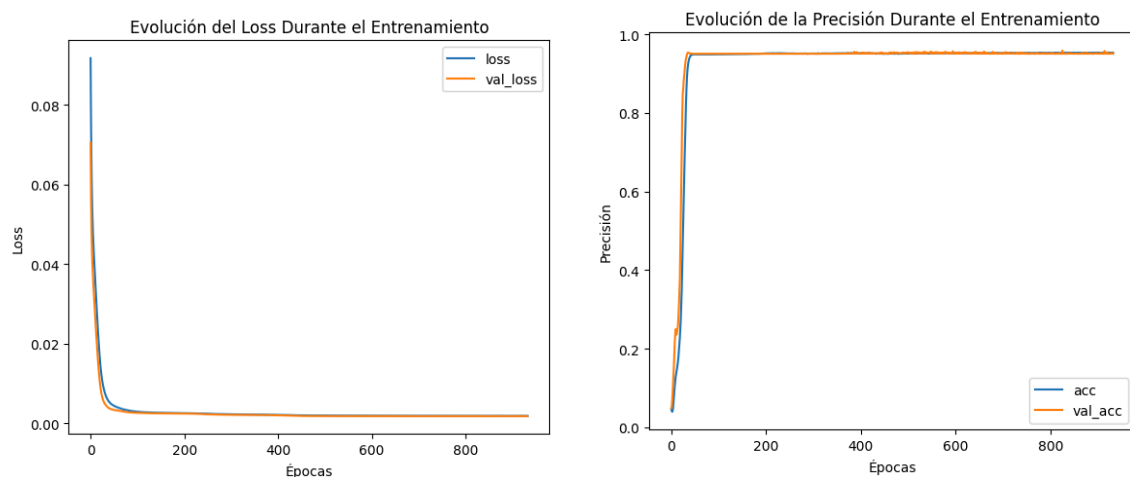
Entrenamiento

- **Número de Épocas:** Se realizaron hasta 1000 iteraciones de entrenamiento, permitiendo un ajuste profundo del modelo a los datos. La cantidad significativa de épocas asegura que el modelo tenga suficiente tiempo para aprender las complejidades de los datos de transacciones.
- **Tamaño del Batch:** Se estableció en 128, un tamaño de lote que equilibra la eficiencia computacional y la estabilidad del aprendizaje. Un batch size adecuado permite una actualización de pesos más estable y un entrenamiento más rápido.
- **Callback - Early Stopping:** Implementado para prevenir el sobreajuste y mejorar la generalización. El entrenamiento se detiene automáticamente si el val_loss no mejora después de un número definido de épocas (patience). Además, restaura los mejores pesos encontrados durante el entrenamiento, asegurando que el modelo mantenga el mejor rendimiento posible en los datos de validación.
- **Objetivo del Entrenamiento:** El enfoque principal es minimizar el error de reconstrucción, medido a través del mse (error cuadrático medio) entre los datos de entrada y sus reconstrucciones. Este objetivo alinea el proceso de aprendizaje del autoencoder con la detección eficaz de anomalías, ya que las transacciones fraudulentas deberían tener un mayor error de reconstrucción.

Curvas de Entrenamiento

En las curvas de entrenamiento se observa, por un lado, un descenso inicial pronunciado de la pérdida tanto en entrenamiento como en validación, seguido de una fase de disminución progresivamente más lenta, que da lugar a una “cola” casi plana. Este patrón indica que el modelo aprende de forma estable en las primeras épocas y posteriormente entra en una zona de convergencia, en la que las mejoras adicionales son cada vez más reducidas.

De forma coherente, las curvas de precisión (accuracy) muestran una tendencia creciente al inicio del entrenamiento, con incrementos más marcados en las primeras épocas y una estabilización posterior en torno a un valor prácticamente constante para entrenamiento y validación. La ausencia de divergencias significativas entre ambas curvas (loss/val_loss y acc/val_acc) o de oscilaciones bruscas en validación sugiere que no se aprecia un sobreajuste y que el modelo alcanza un comportamiento estable y consistente en el tramo final del entrenamiento.

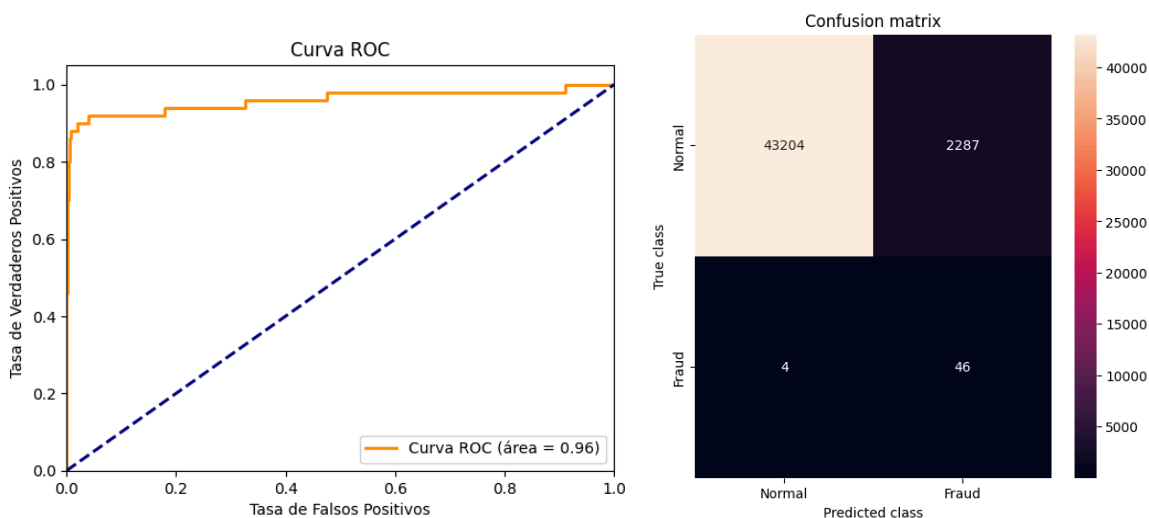


(Figura 5.1: Curvas de Entrenamiento - Modelo AE- Sin Balanceo de Datos)

Evaluación del Autoencoder en el Conjunto de Validación

Al aplicar el modelo de Autoencoder al conjunto de validación, se han obtenido resultados significativos que destacan la capacidad del modelo para identificar y diferenciar entre transacciones normales y fraudulentas. A continuación, se presentan los detalles clave de esta evaluación:

- **Umbral para la Detención de Outliers:** Se estableció en 0.00364. Este valor se utilizó como límite para clasificar una transacción como normal o fraudulenta basándonos en el error de reconstrucción MSE obtenido.
- **Error de reconstrucción medio (MSE):** En el conjunto de validación, el error medio fue de 0.0017. Este bajo error medio indica que el modelo es capaz de reconstruir con precisión la mayoría de las transacciones normales, mientras que las transacciones fraudulentas presentan un error mayor, facilitando su identificación.
- **Área bajo la Curva ROC (AUC):** El modelo alcanzó una área bajo la curva ROC de 0.96, demostrando una excelente capacidad para distinguir entre clases. Un valor cercano a 1 indica un rendimiento de clasificación superior.
- **Resultados de Clasificación:**
 - **Precisión:** Del total de transacciones clasificadas como fraudulentas, el 1.97% fueron identificaciones correctas. Aunque puede parecer bajo, es importante considerar el contexto sobre la detección de fraude.
 - **Sensibilidad (Recall):** La capacidad del modelo para detectar transacciones fraudulentas fue del 92.00%, lo que indica que casi todas las transacciones fraudulentas fueron correctamente identificadas como tales.
 - **Especificidad:** La precisión en la identificación de transacciones normales fue del 94.97%, demostrando la habilidad del modelo para reconocer la mayoría de las transacciones legítimas sin clasificarlas erróneamente como fraudulentas.



(Figura 5.2: Resultados de Clasificación - Modelo AE- Sin Balanceo de Datos)

Modelo Autoencoder Variacional

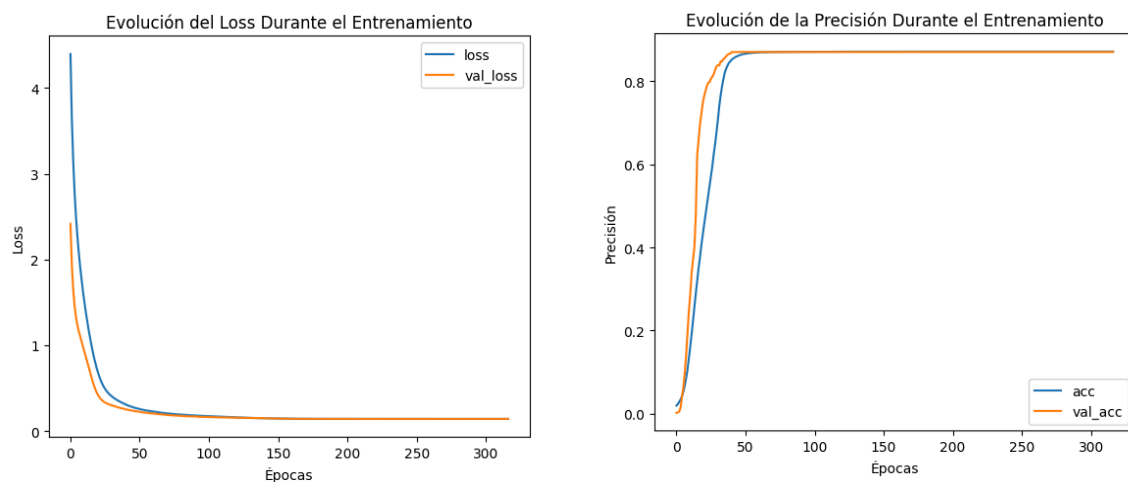
Entrenamiento

- **Número de Épocas:** Se realizaron hasta 317 iteraciones de entrenamiento, permitiendo un ajuste profundo del modelo a los datos. La cantidad significativa de épocas asegura que el modelo tenga suficiente tiempo para aprender las complejidades de los datos de transacciones.
- **Tamaño del Batch:** Se estableció en 256, un tamaño de lote que equilibra la eficiencia computacional y la estabilidad del aprendizaje. Un batch size adecuado permite una actualización de pesos más estable y un entrenamiento más rápido.
- **Callback - Early Stopping:** Implementado para prevenir el sobreajuste y mejorar la generalización. El entrenamiento se detiene automáticamente si el val_loss no mejora después de un número definido de épocas (patience). Además, restaura los mejores pesos encontrados durante el entrenamiento, asegurando que el modelo mantenga el mejor rendimiento posible en los datos de validación.
- **Objetivo del Entrenamiento:** El enfoque principal es minimizar el error de reconstrucción, medido a través del mse (error cuadrático medio) entre los datos de entrada y sus reconstrucciones. Este objetivo alinea el proceso de aprendizaje del autoencoder con la detección eficaz de anomalías, ya que las transacciones fraudulentas deberían tener un mayor error de reconstrucción.

Curvas de Entrenamiento:

En la evolución del loss se aprecia un descenso muy marcado en las primeras épocas, seguido de una reducción progresivamente más suave hasta alcanzar una fase de casi estabilización, sin oscilaciones bruscas ni incrementos relevantes en la pérdida de validación. La proximidad entre las curvas de loss y val_loss sugiere que el modelo aprende de forma estable y que, en este tramo, no se observan indicios claros de sobreajuste.

De manera coherente, las curvas de precisión (acc y val_acc) muestran un aumento rápido al inicio del entrenamiento, con una mejora notable en las primeras decenas de épocas, para después entrar también en una zona de meseta, donde la precisión se mantiene prácticamente constante. La similitud entre las curvas de entrenamiento y validación, junto con la ausencia de oscilaciones significativas, indica que el modelo alcanza un comportamiento robusto y consistente en términos de capacidad predictiva.

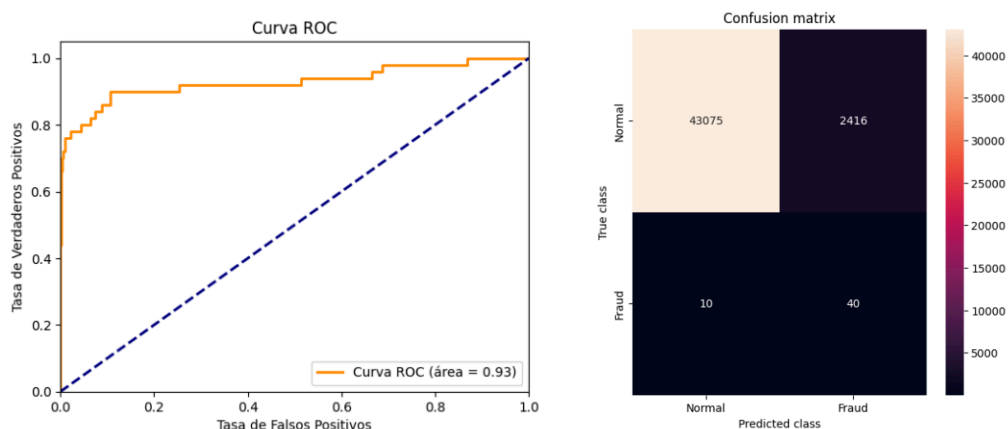


(Figura 5.3: Curvas de Entrenamiento - Modelo VAE - Sin Balanceo de Datos)

Resultados del Entrenamiento con el conjunto de Validación:

Al evaluar nuestro modelo de Autoencoder Variacional (VAE) en el conjunto de validación, he obtenido métricas importantes que indican su desempeño en la detección de transacciones fraudulentas. A continuación, se detallan los resultados clave de esta evaluación:

- **Umbral para la Detección de Outliers:** El umbral para la detección de outliers se estableció en 0.017. Este valor se utiliza para distinguir entre transacciones normales y fraudulentas en función del error de reconstrucción MSE obtenido.
- **Error de reconstrucción medio (MSE):** En el conjunto de validación, el error medio de reconstrucción fue de 0.0107. Este valor indica la discrepancia entre los datos originales y sus reconstrucciones, lo que ayuda en la identificación de transacciones anómalas.
- **Área bajo la Curva ROC (AUC):** El modelo alcanzó un valor de ROC de 0.93, lo que indica una capacidad sólida para discriminar entre transacciones normales y fraudulentas.
- **Resultados de Clasificación:**
 - **Precisión:** Del total de transacciones etiquetadas como fraudulentas, solo el 1.63% fueron identificadas correctamente como tales. Esto sugiere que, aunque el modelo puede detectar algunas transacciones fraudulentas, también clasifica incorrectamente muchas transacciones normales como fraudulentas.
 - **Sensibilidad (Recall):** La sensibilidad del modelo, es decir, su capacidad para detectar transacciones fraudulentas, fue del 80.00%. Esto indica que el modelo pudo identificar correctamente el 80.00% de todas las transacciones fraudulentas presentes en el conjunto de datos.
 - **Especificidad:** La especificidad del modelo, que representa su habilidad para identificar transacciones normales correctamente, fue del 94.69%. Esto indica que el modelo fue eficaz en reconocer la mayoría de las transacciones legítimas sin clasificarlas erróneamente como fraudulentas.



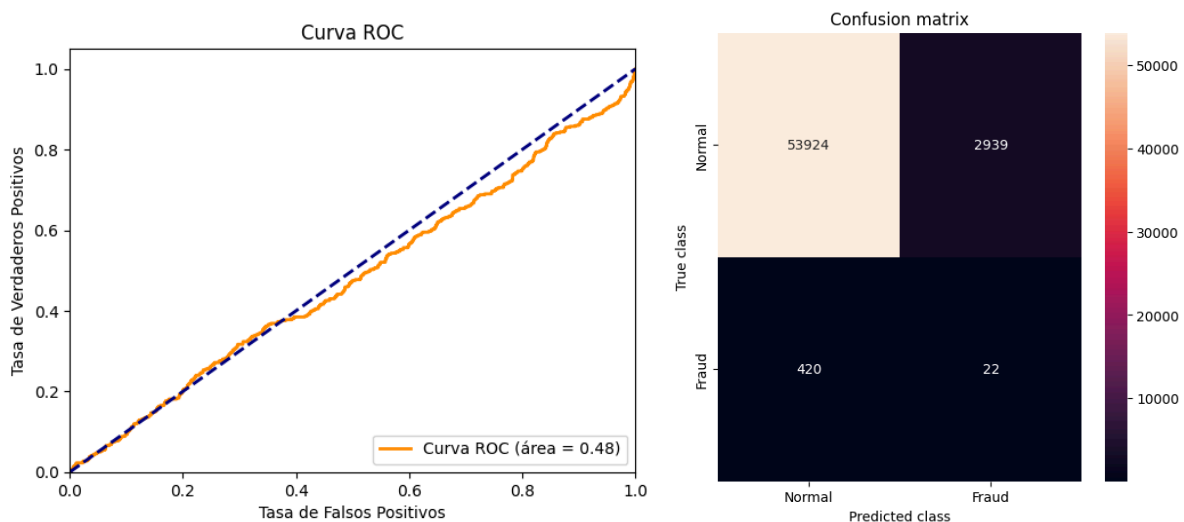
(Figura 5.4: Resultados de Clasificación - Modelo VAE - Sin Balanceo de Datos)

5.1.3 Comparativa de resultados.

PCA

Evaluación de Rendimiento de PCA Tras implementar PCA y evaluar su rendimiento en la detección de anomalías, observamos los siguientes resultados:

- **Curva ROC:** Un valor de 0.48 sugiere un desempeño cercano al azar. Idealmente, buscamos valores más cercanos a 1, lo que indica una mayor capacidad de discriminación entre clases.
- **Precisión:** La precisión extremadamente baja del 0.74% indica que, de las transacciones identificadas como fraudulentas, solo un pequeño porcentaje fue clasificado correctamente. Este resultado sugiere una alta tasa de falsos positivos.
- **Sensibilidad (Recall):** Una sensibilidad del 4.98% significa que solo una pequeña fracción de las transacciones fraudulentas reales fueron detectadas. Este es un indicador crítico en aplicaciones donde es crucial capturar la mayor cantidad posible de eventos anómalos, como en el fraude de tarjetas de crédito.
- **Especificidad:** Aunque la especificidad es relativamente alta 94.83%, indicando una buena identificación de transacciones normales, este resultado no compensa la baja capacidad del modelo para identificar fraudes.

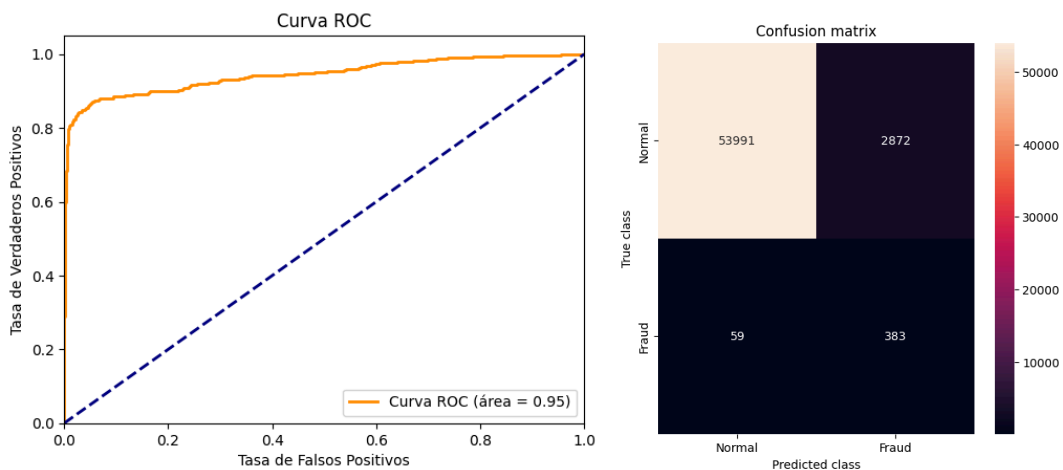


(Figura 5.5: Resultados Test - Modelo PCA - Sin Balanceo de Datos)

Modelo Autoencoder

El Autoencoder se evalúa sobre el conjunto de test manteniendo la prevalencia real de fraude, con el objetivo de analizar su capacidad de generalización a datos no vistos y su rendimiento en un entorno operativo realista.

- **Umbral para la detección de outliers:** el umbral fijado es 0,0036, utilizado como límite de decisión sobre el MSE de reconstrucción para clasificar las transacciones como normales o potencialmente fraudulentas.
- **Error de reconstrucción medio (MSE):** el MSE medio en test es 0,0019, lo que indica que la mayoría de las transacciones legítimas se reconstruyen con una discrepancia reducida respecto a los datos originales, mientras que las transacciones anómalas tienden a presentar errores más elevados, facilitando su identificación a partir de este criterio.
- **Área bajo la Curva ROC (AUC):** El modelo ha alcanzado un AUC de 0.95, lo que indica una excelente capacidad para diferenciar entre transacciones fraudulentas y legítimas. Un AUC cercano a 1 sugiere que el modelo es altamente efectivo.
- **Resultados de Clasificación:**
 - **Precisión:** Un 11.77% de las transacciones identificadas como fraudulentas fueron clasificaciones correctas. Este aumento en la precisión, comparado con los resultados en el conjunto de validación, sugiere que el modelo es aún más efectivo cuando se enfrenta a datos nuevos.
 - **Sensibilidad (Recall):** La sensibilidad o capacidad del modelo para identificar correctamente las transacciones fraudulentas fue de 86.65%. Esto indica que el modelo es capaz de capturar una gran mayoría de las actividades fraudulentas, minimizando los casos no detectados.
 - **Especificidad:** La especificidad del modelo, o su habilidad para identificar correctamente las transacciones legítimas, se mantuvo robusta en 94.95%. Este valor asegura que la mayoría de las transacciones legítimas no son erróneamente marcadas como fraudulentas.

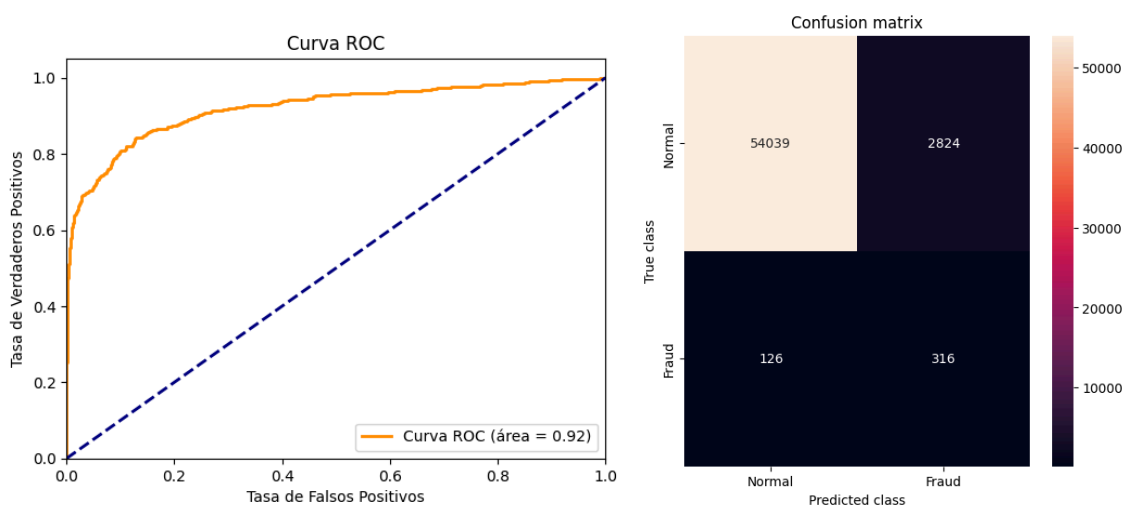


(Figura 5.6: Resultados Test - Modelo AE - Sin Balanceo de Datos)

Modelo Autoencoder Variacional

La evaluación de nuestro modelo de Autoencoder Variacional en el conjunto de test proporciona información crucial sobre su rendimiento en un entorno de datos diverso y realista.

- **Umbral para la Detección de Outliers:** El umbral para la detección de outliers se estableció en 0.017. Este valor se utiliza para clasificar las transacciones como normales o fraudulentas basándose en el error de reconstrucción MSE obtenido.
- **Error de reconstrucción medio (MSE):** En el conjunto de test, el error medio de reconstrucción fue de 0.0109. Este valor indica la discrepancia entre los datos originales y sus reconstrucciones, lo que ayuda en la identificación de transacciones anómalas.
- **Área bajo la Curva ROC (AUC):** El modelo alcanzó un valor de ROC de 0.92, lo que sugiere una buena capacidad para distinguir entre transacciones normales y fraudulentas.
- **Resultados de Clasificación:**
 - **Precisión:** De todas las transacciones etiquetadas como fraudulentas, solo el 10.06% fueron identificadas correctamente como tales. Esto indica que el modelo tiende a clasificar incorrectamente muchas transacciones normales como fraudulentas.
 - **Sensibilidad (Recall):** La capacidad del modelo para detectar transacciones fraudulentas fue del 71.49%. Esto indica que el modelo pudo identificar correctamente el 71.49% de todas las transacciones fraudulentas presentes en el conjunto de datos.
 - **Especificidad:** La especificidad del modelo, que representa su habilidad para identificar transacciones normales correctamente, fue del 95.03%. Esto indica que el modelo fue eficaz en reconocer la mayoría de las transacciones legítimas sin clasificarlas erróneamente como fraudulentas.



(Figura 5.7: Resultados Test - Modelo VAE - Sin Balanceo de Datos)

5.2 Con Balanceo de Datos(SMOTE)

En este segundo escenario el modelo se entrena sobre un conjunto balanceado con SMOTE, de modo que la clase minoritaria(fraude) gana presencia en el entrenamiento sin alterar validación ni test. El objetivo es contrastar cómo varía el aprendizaje cuando el autoencoder y su variante variacional se exponen a ejemplos sintéticos de la clase 1, manteniendo el mismo protocolo de preprocesado, normalización y evaluación.

5.2.1 Aplicación de modelos de aprendizaje automático.

Modelo Autoencoder

El autoencoder se organiza como una arquitectura secuencial con dos bloques simétricos y un cuello de botella regularizado. En el codificador, una capa densa de entrada recibe el vector de características y aplica ELU para preservar la no linealidad y estabilidad frente a valores negativos. A continuación, la Batch Normalization estabiliza la distribución de activaciones y acelera el entrenamiento, y un Dropout del 10% introduce ruido controlado para contener el sobreajuste. La compresión se profundiza con capas densas intermedias de 64, 32, 16 neuronas todas con ELU, tras las cuales se repite el bloque de Batch Normalization y Dropout del 10% como refuerzo de la regularización. Una capa densa de 8 unidades compacta la representación inmediatamente antes del cuello de botella.

La decodificación invierte el proceso con capas Densas de 8, 16, 32, 64 neuronas todas utilizando ELU para expandir gradualmente la latente hacia el espacio original, incluyendo de nuevo Batch Normalization y Dropout del 10% en la parte final para estabilizar el aprendizaje y reducir el riesgo de overfitting. La capa de salida es densa con activación sigmoide y tamaño igual a 30, responsable de la reconstrucción. El optimizador seleccionado es Adam con una tasa de aprendizaje 0.000005, valor bajo que favorece una convergencia suave en alta dimensionalidad y paisajes no convexos.

Modelo Autoencoder Variacional

El VAE adopta una arquitectura funcional con codificador, cuello de botella probabilístico y decodificador, integrando normalización, dropout y regularización L1. El encoder recibe tensores y comienza con un bloque Denso con ELU para preservar la no linealidades y regularización Kernel del tipo L1, seguido de Batch Normalization para estabilizar distribuciones internas y Dropout del 10% para reducir sobreajuste. La profundización continúa con capas Densas intermedias de 64 y 32 unidades Intercaladas opr capas de Batch Normalization, aprovechando ELU para mejorar la representación de valores negativos y la velocidad de convergencia.

El cuello de botella define una dimensión latente de 4 neuronas mediante z_mean y z_log_var). La representación final z se obtiene por reparametrización, permitiendo la propagación del error hacia atrás, desde la capa de salida hasta las capas de entrada.

El decodificador toma z y reconstruye progresivamente mediante capas Densas de 32 y 64 unidades seguidas cada una de Batch Normalization para estabilizar distribuciones internas, en este caso optando por ReLU para favorecer activaciones dispersas y estables, hasta la capa de salida Densa que restituye el vector original. La pérdida total del VAE combina el MSE de reconstrucción y la divergencia KL entre la distribución latente aprendida y el prior, inyectada en el grafo mediante una capa personalizada.. Para la optimización se emplea Adam con tasa de aprendizaje 0.000005, adecuada para el equilibrio entre el término de reconstrucción y la regularización probabilística.

5.2.2 Evaluación y validación de modelos de aprendizaje automático.

En el escenario con balanceo de datos mediante SMOTE se evalúa la capacidad de los modelos de reconstrucción cuando el conjunto de entrenamiento incorpora una proporción reforzada de transacciones fraudulentas, mientras que los conjuntos de validación y test mantienen la prevalencia real. De nuevo, ambos modelos comparten el mismo protocolo: datos previamente normalizados, entrenamiento monitorizado con *early stopping* y decisión operativa basada en un umbral fijado a partir del percentil 95 del error de reconstrucción en el conjunto de entrenamiento balanceado. A continuación se describe el proceso de entrenamiento y los resultados obtenidos en validación para el Autoencoder (AE) y el Autoencoder Variacional (VAE) bajo este esquema.

Modelo Autoencoder

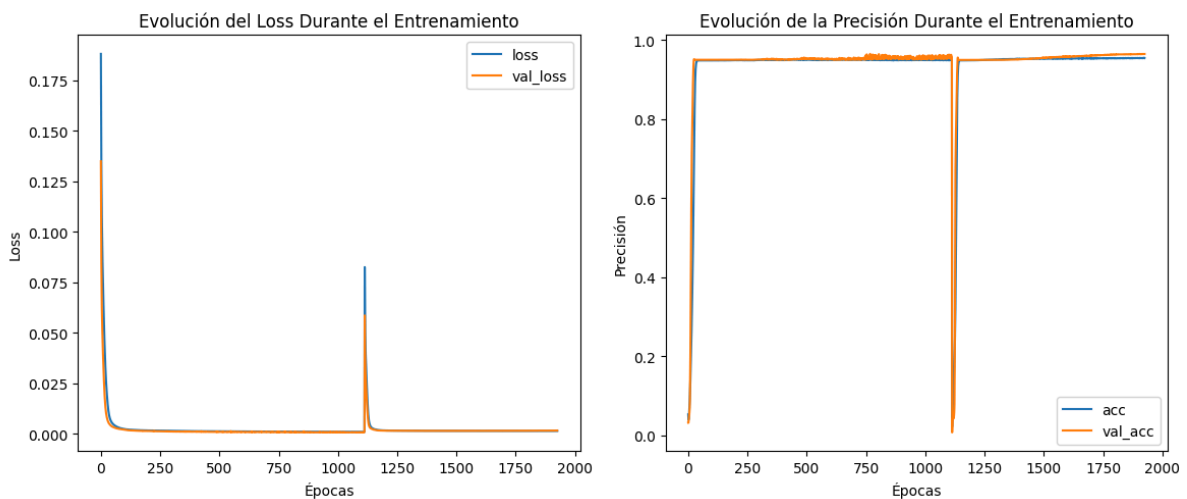
Entrenamiento

- **Número de Épocas:** Se realizaron hasta 2500 iteraciones de entrenamiento, permitiendo un ajuste profundo del modelo a los datos. La cantidad significativa de épocas asegura que el modelo tenga suficiente tiempo para aprender las complejidades de los datos de transacciones.
- **Tamaño del Batch:** Se estableció en 128, un tamaño de lote que equilibra la eficiencia computacional y la estabilidad del aprendizaje. Un batch size adecuado permite una actualización de pesos más estable y un entrenamiento más rápido.
- **Callback - Early Stopping:** Implementado para prevenir el sobreajuste y mejorar la generalización. El entrenamiento se detiene automáticamente si el `val_loss` no mejora después de un número definido de épocas (`patience`). Además, restaura los mejores pesos encontrados durante el entrenamiento, asegurando que el modelo mantenga el mejor rendimiento posible en los datos de validación.
- **Objetivo del Entrenamiento:** El enfoque principal es minimizar el error de reconstrucción, medido a través del mse (error cuadrático medio) entre los datos de entrada y sus reconstrucciones. Este objetivo alinea el proceso de aprendizaje del autoencoder con la detección eficaz de anomalías, ya que las transacciones fraudulentas deberían tener un mayor error de reconstrucción.

Curvas de Entrenamiento:

En las curvas de entrenamiento se observa, en primer lugar, un descenso rápido de la pérdida en las primeras épocas, acompañado de un incremento acusado de la precisión, hasta alcanzar valores próximos a 1 tanto en entrenamiento como en validación. A partir de ese punto, el modelo entra en una fase de estabilización, en la que loss y val_loss se mantienen muy próximas a cero y las curvas de acc y val_acc oscilan ligeramente alrededor de un valor casi constante.

En torno a la mitad del entrenamiento aparece un pico aislado simultáneo en la pérdida y en la precisión, visible tanto en entrenamiento como en validación. Este comportamiento puede asociarse a una actualización de pesos especialmente grande (por ejemplo, debida a un minibatch poco representativo o a la interacción con el programador de la tasa de aprendizaje), pero el hecho de que las curvas vuelvan rápidamente a los niveles previos indica que se trata de una perturbación transitoria y no de un cambio estructural en el modelo. En conjunto, la trayectoria sugiere un proceso de aprendizaje estable y robusto, sin sobreajuste sostenido y con capacidad de recuperación frente a pequeñas inestabilidades durante el entrenamiento.

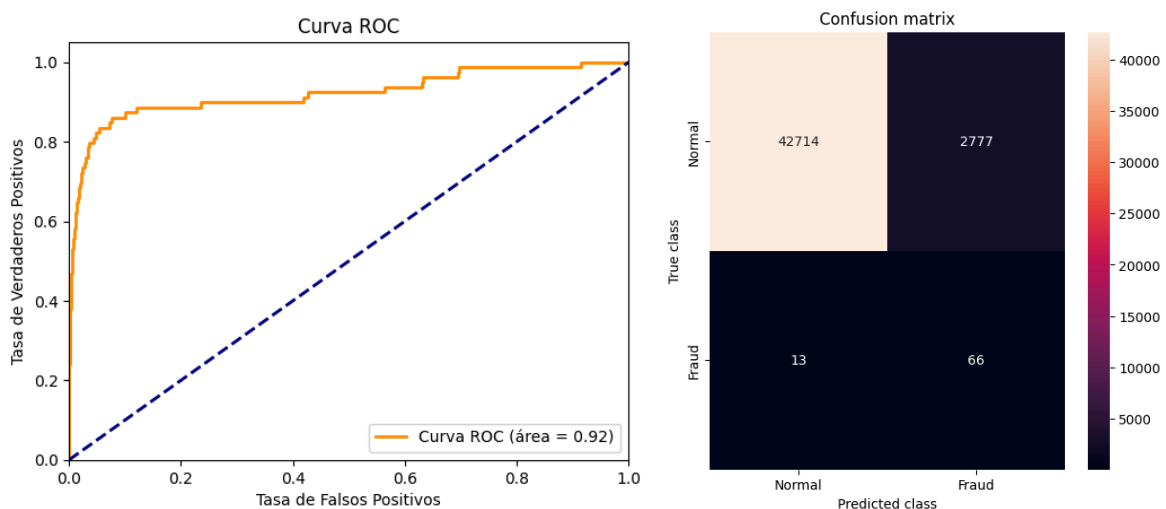


(Figura 5.8: Curvas de Entrenamiento - Modelo AE - SMOTE)

Evaluación del Autoencoder en el Conjunto de Validación

Al aplicar nuestro modelo de Autoencoder al conjunto de validación con datos balanceados. Los resultados obtenidos permiten valorar su capacidad para discriminar entre transacciones normales y fraudulentas.

- **Umbral para la Detección de Outliers:** 0.007. utilizado como límite de decisión sobre el MSE de reconstrucción. Las observaciones con un error igual o superior a este valor se etiquetan como potencialmente fraudulentas.
- **Error de reconstrucción medio (MSE):** el MSE medio es 0,005, inferior al umbral establecido. Esto indica que la mayoría de las transacciones normales se reconstruyen con un error reducido, mientras que las transacciones fraudulentas tienden a concentrarse en la zona de errores más altos, lo que facilita su separación.
- **Área bajo la curva ROC (AUC):** el modelo alcanza un ROC-AUC de 0,92, lo que refleja una buena capacidad global de discriminación entre clase normal y clase fraudulenta para el conjunto de umbrales posibles.
- **Resultados de Clasificación:**
 - **Precisión (PPV):** 2,32%; es decir, de todas las transacciones señaladas como fraudulentas, un 2,32% resultan ser fraudes reales. Dado el fuerte desbalance de clases, este valor es coherente con el contexto y debe interpretarse considerando la muy baja prevalencia de la clase positiva.
 - **Sensibilidad (Recall):** 83,54%; el modelo detecta la gran mayoría de las transacciones fraudulentas presentes en validación, reduciendo el porcentaje de fraudes no identificados.
 - **Especificidad:** 93,90%; una proporción elevada de transacciones legítimas se clasifica correctamente como normal, conteniendo la tasa de falsas alarmas sobre clientes no fraudulentos.



(Figura 5.9: Resultados de Clasificación - Modelo AE - SMOTE)

Modelo Autoencoder Variacional

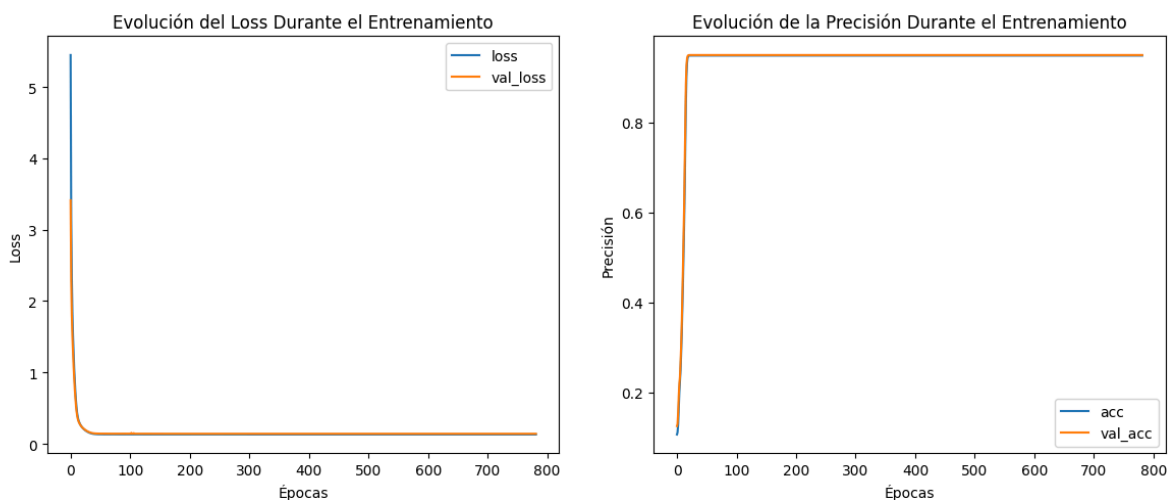
Entrenamiento

- **Número de Épocas:** Se realizaron hasta 2500 iteraciones de entrenamiento, permitiendo un ajuste profundo del modelo a los datos. La cantidad significativa de épocas asegura que el modelo tenga suficiente tiempo para aprender las complejidades de los datos de transacciones.
- **Tamaño del Batch:** Se estableció en 128, un tamaño de lote que equilibra la eficiencia computacional y la estabilidad del aprendizaje. Un batch size adecuado permite una actualización de pesos más estable y un entrenamiento más rápido.
- **Callback - Early Stopping:** Implementado para prevenir el sobreajuste y mejorar la generalización. El entrenamiento se detiene automáticamente si el val_loss no mejora después de un número definido de épocas (patience). Además, restaura los mejores pesos encontrados durante el entrenamiento, asegurando que el modelo mantenga el mejor rendimiento posible en los datos de validación.
- **Objetivo del Entrenamiento:** El enfoque principal es minimizar el error de reconstrucción, medido a través del mse (error cuadrático medio) entre los datos de entrada y sus reconstrucciones. Este objetivo alinea el proceso de aprendizaje del autoencoder con la detección eficaz de anomalías, ya que las transacciones fraudulentas deberían tener un mayor error de reconstrucción.

Curvas de Entrenamiento

En las curvas de entrenamiento se aprecia, en primer lugar, un descenso muy acusado de la pérdida durante las primeras épocas, tanto en entrenamiento como en validación, seguido de una fase en la que loss y val_loss continúan disminuyendo de forma mucho más suave hasta formar una “cola” prácticamente plana. Este patrón indica que el modelo aprende rápido al inicio y posteriormente entra en una zona de convergencia, sin oscilaciones bruscas ni divergencias entre ambas curvas, lo que sugiere ausencia de sobreajuste relevante.

De forma coherente, las curvas de precisión (acc y val_acc) muestran un incremento rápido en las primeras épocas, hasta alcanzar valores cercanos a 1, tras lo cual se mantienen estables a lo largo del resto del entrenamiento. La similitud entre las trayectorias de entrenamiento y validación refuerza la idea de un comportamiento robusto y consistente, con una capacidad de clasificación que se consolida pronto y se preserva hasta el final del proceso de ajuste.

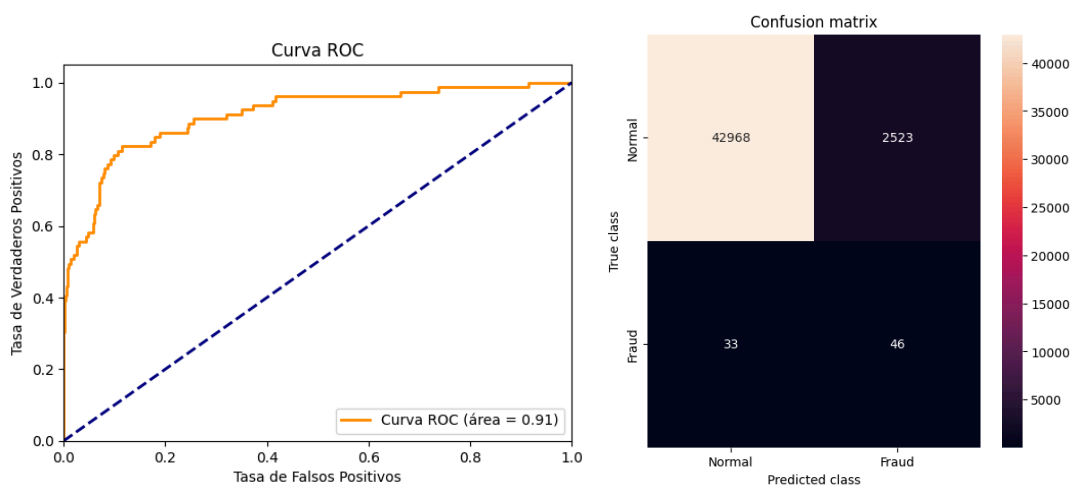


(Figura 5.10: Curvas de Entrenamiento - Modelo VAE - SMOTE)

Evaluación del Autoencoder Variacional en el Conjunto de Validación

El Autoencoder Variacional (VAE) se evalúa sobre el conjunto de validación con prevalencia real de fraude, con el objetivo de evaluar su capacidad para discriminar entre transacciones normales y fraudulentas a partir del error de reconstrucción.

- **Umbral para la detección de outliers:** se fija en 0,024, utilizando el MSE de reconstrucción como puntuación de anomalía. Las observaciones cuyo error es igual o superior a este umbral se etiquetan como potencialmente fraudulentas.
- **Error de reconstrucción medio (MSE):** el MSE medio en validación es 0,0159, valor inferior al umbral τ . Esto indica que el modelo reconstruye con un error relativamente bajo la mayor parte de las transacciones normales, mientras que las transacciones fraudulentas tienden a concentrarse en la región de errores más elevados, lo que facilita su diferenciación.
- **Área bajo la curva ROC (AUC):** el modelo alcanza un ROC-AUC de 0,91, lo que refleja una capacidad discriminativa sólida para separar ambas clases a lo largo del conjunto de posibles umbrales.
- **Resultados de Clasificación:**
 - **Precisión (PPV):** 1,79%; es decir, de todas las transacciones señaladas como fraude por el modelo, un 1,79% corresponden efectivamente a fraudes reales. Aunque este valor es reducido, resulta coherente con un contexto de prevalencia muy baja de la clase positiva.
 - **Sensibilidad (Recall):** 58,23%; el modelo identifica correctamente algo más de la mitad de las transacciones fraudulentas presentes en el conjunto de validación, dejando todavía una proporción relevante sin detectar.
 - **Especificidad:** 94,45%; una fracción muy elevada de las transacciones legítimas se clasifica correctamente como normales, lo que contribuye a contener la tasa de falsas alarmas sobre clientes no fraudulentos.



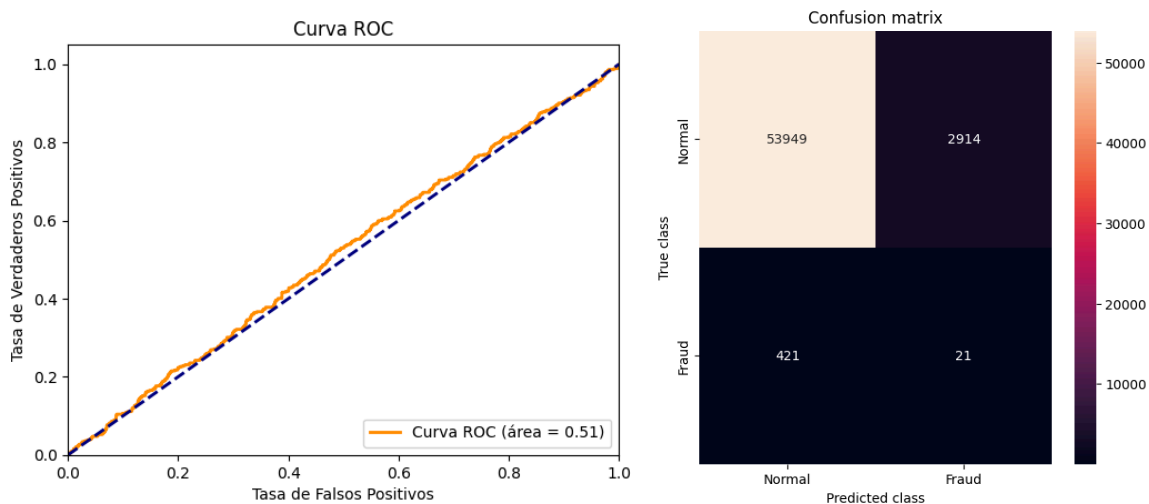
(Figura 5.11: Resultados de Clasificación - Modelo VAE - SMOTE)

5.2.3 Comparativa de resultados.

PCA

Evaluación de Rendimiento de PCA Tras implementar PCA y evaluar su rendimiento en la detección de anomalías, observamos los siguientes resultados:

- **Curva ROC:** Un valor de 0.51 indica un rendimiento cercano a la aleatoriedad. Lo que buscamos son valores más cercanos a 1, lo que indica una mayor capacidad de discriminación entre clases.
- **Precisión:** La precisión extremadamente baja del 0.72% indica que, de las transacciones identificadas como fraudulentas, solo un pequeño porcentaje fue clasificado correctamente. Este resultado sugiere una alta tasa de falsos positivos.
- **Sensibilidad (Recall):** Una sensibilidad del 4.75% significa que solo es capaz de identificar una cantidad muy pequeña de las transacciones fraudulentas reales.
- **Especificidad:** Aunque la especificidad es relativamente alta 94.88%, mostrando que identifica la gran parte de las transacciones correctas, este resultado no compensa la baja capacidad del modelo para identificar fraudes.

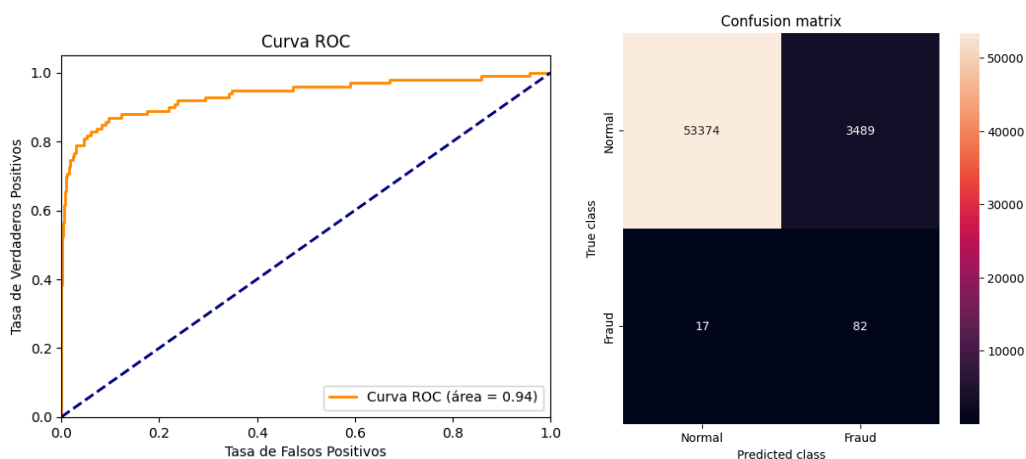


(Figura 5.12: Resultados Test - Modelo PCA - SMOTE)

Modelo Autoencoder

En esta sección se evalúa el modelo Autoencoder sobre el conjunto de test, que no ha intervenido en ninguna fase del ajuste, con el fin de medir su rendimiento en condiciones de generalización. El análisis de las métricas permite determinar hasta qué punto el modelo resulta adecuado para su uso en un entorno de detección de fraude en producción.

- **Umbral para la detección de outliers:** se utiliza el umbral .007, definido a partir del error de reconstrucción en entrenamiento. Toda transacción cuyo MSE de reconstrucción sea igual o superior a este valor se etiqueta como potencialmente fraudulenta.
- **Error de reconstrucción medio (MSE):** el MSE medio en test es 0.005, por debajo del umbral τ . Esto indica que la mayoría de las transacciones normales se reconstruyen con un error reducido, mientras que las transacciones fraudulentas tienden a situarse en la región de errores más elevados, lo que respalda el uso del MSE como puntuación de anomalía.
- **Área bajo la curva ROC (AUC):** el modelo alcanza un ROC-AUC de 0.94, lo que refleja una alta capacidad discriminativa global entre transacciones legítimas y fraudulentas (véase curva ROC correspondiente).
- **Resultados de clasificación al umbral τ :**
 - **Precisión (PPV):** 2.3%; de todas las alertas de fraude emitidas por el modelo, un 2.3% corresponden efectivamente a fraudes reales. Este valor, aunque modesto, es coherente con la muy baja prevalencia de la clase positiva.
 - **Sensibilidad (Recall):** 82.83%; el modelo detecta la mayoría de las transacciones fraudulentas presentes en el conjunto de test, reduciendo el número de fraudes no identificados.
 - **Especificidad:** 93.86%; una proporción muy elevada de las transacciones legítimas se clasifica correctamente como normales, lo que contribuye a contener la tasa de falsas alarmas.

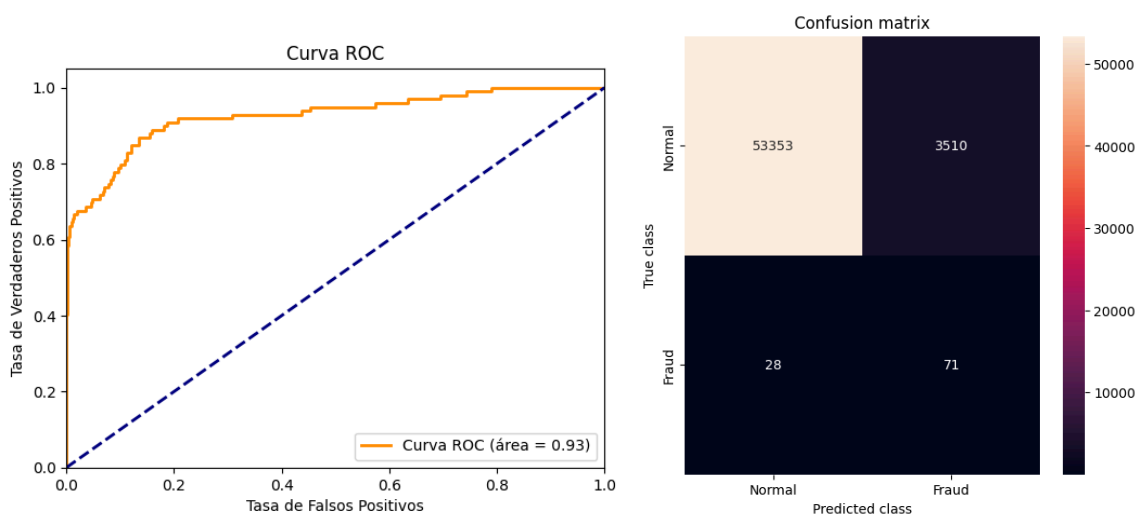


(Figura 5.13: Resultados Test - Modelo AE - SMOTE)

Modelo Autoencoder Variacional

El modelo Autoencoder Variacional se evalúa sobre el conjunto de test manteniendo la prevalencia real de fraude, con el propósito de analizar su capacidad de generalización frente a datos no vistos. Esta evaluación permite valorar si el comportamiento observado en entrenamiento y validación se mantiene en un escenario operativo realista.

- **Umbral para la detección de outliers:** 0.0138. Toda transacción cuyo MSE de reconstrucción es igual o superior a este valor se clasifica como potencialmente fraudulenta.
- **Error de reconstrucción medio (MSE):** 0.005, claramente por debajo del umbral τ . Esto indica que la mayoría de las transacciones normales se reconstruyen con una discrepancia reducida respecto a los datos originales, mientras que las transacciones anómalas tienden a concentrarse en la zona de errores más elevados.
- **Área bajo la curva ROC (AUC):** 0.93. La curva ROC refleja una buena capacidad discriminativa global del modelo para separar transacciones legítimas de fraudulentas a lo largo de un amplio rango de umbrales.
- **Resultados de Clasificación:**
 - **Precisión (PPV):** 1.98%; de todas las alertas de fraude generadas por el modelo, un 1.98% corresponden efectivamente a fraudes reales. Este valor, aunque reducido, es coherente con el contexto de prevalencia muy baja de la clase fraudulenta.
 - **Sensibilidad (Recall):** 71.72%; el modelo identifica correctamente algo más de siete de cada diez transacciones fraudulentas presentes en el conjunto de test, dejando aún una fracción relevante sin detectar.**Especificidad:** 93.78%; una proporción muy elevada de las transacciones legítimas se clasifica correctamente como normal, lo que contribuye a mantener controlada la tasa de falsas alarmas sobre clientes no fraudulentos.



(Figura 5.14: Resultados Test - Modelo VAE - SMOTE)

Capítulo 6

Conclusiones

Síntesis respecto a objetivos.

El objetivo general del trabajo fue abordar la detección de fraude en transacciones financieras mediante técnicas de aprendizaje automático con énfasis en modelos de autoencoder, tomando PCA como línea base y comparando con un autoencoder estándar (AE) y un autoencoder variacional (VAE) bajo dos escenarios de evaluación: datos sin balanceo y datos balanceados con SMOTE.

En primer lugar, se planteó como objetivo metodológico la construcción de un pipeline reproducible para la preparación, partición y normalización de los datos. Este pipeline debía estar claramente documentado y diferenciar explícitamente las dos casuísticas (escenario sin balanceo y escenario con SMOTE), incluyendo el uso de semillas fijas y comentarios en el código para reducir la aleatoriedad y facilitar la replicabilidad del estudio.

En segundo lugar, se buscó entrenar y evaluar varios modelos de autoencoder sobre el conjunto de datos disponible, utilizando el error de reconstrucción como puntuación de anomalía. El diseño contempló tanto la evaluación en conjuntos de validación como en conjuntos de test, con el fin de analizar la capacidad de los modelos para distinguir entre transacciones legítimas y fraudulentas, y estudiar el impacto de la elección del umbral sobre las métricas resultantes.

Por último, se propuso comparar de forma sistemática los resultados de los tres modelos considerados (PCA, AE y VAE), con especial atención a métricas clave en problemas de fraude como la ROC-AUC, la sensibilidad, la especificidad y la precisión, y bajo las dos estrategias de preparación de datos (sin balanceo y con SMOTE).

En conjunto, el trabajo cumple estos objetivos: se ha implementado un pipeline claro y reproducible, se han entrenado y evaluado varios modelos de autoencoder sobre un conjunto real de transacciones financieras y se ha realizado una comparación rigurosa con un baseline lineal (PCA) en términos de capacidad de detección de fraude, documentando tanto las fortalezas como las limitaciones de cada enfoque.

Análisis de los resultados:

Desde el punto de vista cuantitativo, los resultados obtenidos permiten diferenciar con claridad el comportamiento de los distintos modelos y escenarios.

En el escenario sin balanceo de datos, PCA actúa como baseline y muestra un rendimiento muy limitado: en el conjunto de test alcanza una ROC-AUC de 0,48, con una sensibilidad del 4,98%, precisión del 0,74% y especificidad del 94,83%. En la práctica, esto se traduce en una capacidad casi nula para detectar fraudes reales (muy bajo recall) y en alertas de muy baja calidad (casi todas falsas alarmas), a pesar de identificar correctamente la mayoría de transacciones normales. Este comportamiento ilustra la insuficiencia de un enfoque lineal para capturar patrones de fraude en un contexto de fuerte desbalance.

En contraste, el Autoencoder (AE) sin balanceo ofrece una separabilidad excelente. En test alcanza una ROC-AUC de 0,95, con sensibilidad del 86,65%, especificidad del 94,95% y precisión del 11,77%. Es decir, detecta una gran parte de las transacciones fraudulentas manteniendo una fracción elevada de transacciones legítimas correctamente clasificadas. En validación (sin balanceo) los resultados son consistentes (ROC-AUC de 0,96, recall del 92,00% y especificidad del 94,97%), lo que indica una buena capacidad de generalización del modelo a datos no vistos.

El Autoencoder variacional (VAE) sin balanceo muestra un rendimiento también sólido, aunque ligeramente inferior al AE al mismo umbral. En el conjunto de test obtiene una ROC-AUC de 0,92, con sensibilidad del 71,49%, especificidad del 95,03% y precisión del 10,06%. En validación sin balanceo, la ROC-AUC es de 0,93, con recall del 80,00% y especificidad del 94,69%. Esto indica que el VAE conserva una capacidad de discriminación alta y mantiene una especificidad muy elevada, pero detecta una proporción algo menor de fraudes en comparación con el AE al mismo punto de corte, a cambio de una reconstrucción más regularizada.

En el escenario con SMOTE, el entrenamiento se realiza sobre un conjunto de datos balanceado, mientras que la evaluación en validación y test se mantiene con prevalencia real. En este contexto, el AE con SMOTE alcanza en test una ROC-AUC de 0,94, con sensibilidad del 82,83%, especificidad del 93,86% y precisión del 2,30%. El VAE con SMOTE obtiene ROC-AUC de 0,93, sensibilidad del 71,72%, especificidad del 93,78% y precisión del 1,98%. Es decir, la capacidad discriminativa global (ROC-AUC) se mantiene alta y muy similar a la del escenario sin balanceo, y la sensibilidad sigue siendo competitiva, especialmente en el AE, pero la precisión desciende de forma notable al entrenar con SMOTE, lo que implica un incremento del número de falsas alarmas por cada fraude detectado.

En validación bajo SMOTE y umbral fijo se observaron métricas alineadas con esta tendencia (recall elevado, especificidad alrededor del 93–94% y precisión en torno al 2,32%), confirmando que el balanceo en entrenamiento no empeora la capacidad de detección, pero tampoco mejora de forma clara el compromiso entre detección y calidad de las alertas cuando la evaluación se mantiene en prevalencia real. En conjunto, los resultados muestran que AE y VAE superan ampliamente a PCA en ambos escenarios, y que el autoencoder estándar ofrece, en general, el mejor equilibrio entre sensibilidad y especificidad, tanto con como sin SMOTE.

Implicaciones y lectura operativa.

- **Autoencoders frente a PCA:** La comparación directa entre PCA y los modelos de autoencoder deja patente la ventaja sustancial de estos últimos en el contexto de detección de fraude. Mientras que PCA se limita a una ROC-AUC cercana al azar (0,48) y a una sensibilidad residual (4,98%), los autoencoders alcanzan ROC-AUC de 0,95 y 0,92 (AE y VAE sin balanceo) y 0,94 y 0,93 (AE y VAE con SMOTE), junto con sensibilidades en el rango 71–82%. Esto significa que los modelos basados en reconstrucción no solo separan mejor las clases en términos globales, sino que además capturan un volumen mucho mayor de fraudes reales, manteniendo al mismo tiempo una especificidad elevada. En un escenario operativo, esta diferencia se traduce en una mejora significativa de la capacidad del sistema para detectar y priorizar transacciones sospechosas.
- **Score de anomalía como herramienta funcional:** El uso del MSE de reconstrucción como puntuación de anomalía resulta operativo y flexible. El umbral de decisión permite ajustar el compromiso entre falsos positivos (FP) y falsos negativos (FN) en función de la política de riesgo: umbrales más bajos aumentan la sensibilidad y la tasa de alertas (adecuado, por ejemplo, para campañas intensivas de contención), mientras que umbrales más altos reducen la carga de revisión a costa de aceptar un mayor número de fraudes no detectados. El hecho de que los modelos mantengan ROC-AUC altas indica que, para un amplio rango de umbrales, existe una compensación manejable entre recall y especificidad, lo que refuerza el papel del error de reconstrucción como score central en un sistema de priorización de operaciones.
- **Precisión modesta:** En términos de precisión, si bien los porcentajes pueden parecer no muy buenos, es crucial considerar el contexto del problema de detección de fraude, donde incluso pequeñas mejoras en la precisión pueden tener un impacto significativo. Más notable aún es la mejora en la sensibilidad, pasando de un 4.98% con la PCA a un 86.65% con el Autoencoder y un 71.49% con el Autoencoder Variacional. Esto indica que los Autoencoders son capaces de detectar una mayor proporción de transacciones fraudulentas reales, un aspecto crítico para minimizar el daño financiero y mejorar la seguridad.
- **Alta especificidad:** La especificidad se mantuvo alta en todos los casos, asegurando que la mayoría de las transacciones legítimas no se marcaran incorrectamente como fraudulentas. Sin embargo, la mejora en la detección de transacciones fraudulentas sin sacrificar significativamente la identificación correcta de las transacciones legítimas subraya la utilidad de los Autoencoders y Autoencoders Variacionales sobre métodos más simples como la PCA.

Limitaciones y alcance del estudio.

- La ausencia del conjunto de datos original y la dependencia en las variables transformadas por PCA plantea dudas sobre si los modelos son óptimos para los datos originales y para la propuesta o objetivo del trabajo. La PCA, si bien efectiva para simplificar los datos, puede no ser la estrategia óptima siempre, especialmente si descarta información sutil pero crucial para identificar fraudes. Además, al trabajar únicamente con datos ya procesados mediante PCA, existe el riesgo de que los resultados estén sesgados por la manera en que se realizó esta transformación inicial. Esto podría limitar la efectividad de los modelos.
- A ello se suma que la precisión (PPV) obtenida se mantiene en valores relativamente bajos, consecuencia directa de que la tasa de referencia es extremadamente reducida de la clase fraudulenta. Esta característica es inherente al dominio y no invalida el uso de los modelos, pero implica que su despliegue real debe ir acompañado de una calibración cuidadosa del umbral y de una adecuada orquestación con los procesos de negocio (priorización de alertas, triaje y revisión manual), para que el volumen de falsas alarmas sea manejable operativamente.

Líneas de trabajo futuro

Aunque los resultados obtenidos son satisfactorios para los objetivos planteados en este Trabajo Final de Máster, se identifican varias líneas de trabajo que podrían explorarse en futuras iteraciones de la investigación para ampliar su alcance y reforzar su aplicabilidad en entornos reales.

En primer lugar, la precisión (PPV) se mantiene en niveles relativamente bajos (entre el 1,6 % y el 11,7 %), lo que implicaría, en un escenario operativo, un volumen de falsas alarmas elevado y potencialmente difícil de gestionar. Una posible extensión consistiría en incorporar una etapa de post-procesado supervisado sobre las alertas generadas por el autoencoder, empleando modelos ligeros como, por ejemplo, un Random Forest u otros clasificadores de baja complejidad. Esta capa adicional podría actuar como filtro de los falsos positivos, manteniendo una alta sensibilidad en la detección de fraude pero reduciendo la carga de revisión manual asociada a las alertas.

En segundo lugar, el estudio se basa íntegramente en variables ya transformadas mediante un PCA de anonimización, lo que limita la posibilidad de realizar ingeniería de variables a partir de atributos originales. Una línea de trabajo futura de especial interés sería replicar la metodología sobre un conjunto de datos con variables en crudo, de forma que pudiera evaluarse hasta qué punto los autoencoders son capaces de aprender representaciones latentes más informativas que las obtenidas por el PCA inicial. Esta comparación permitiría valorar el papel del autoencoder no solo como detector de anomalías, sino también como alternativa de extracción de características frente a enfoques de reducción de dimensionalidad lineales.

Por último, aunque el autoencoder variacional (VAE) representa una técnica más avanzada desde el punto de vista teórico, sus resultados han sido ligeramente inferiores a los del autoencoder estándar en términos de sensibilidad. En este sentido, futuras investigaciones podrían incluir un análisis de sensibilidad más detallado sobre el peso del término de divergencia KL en la función de pérdida, por ejemplo mediante esquemas tipo *beta-VAE* en los que el coeficiente asociado a la KL se incremente de forma gradual durante el entrenamiento. Este tipo de experimentos permitiría estudiar si un control más fino de la regularización latente mejora la fidelidad de la reconstrucción y, en consecuencia, la capacidad del VAE para discriminar entre transacciones normales y fraudulentas.

En conjunto, estas líneas de trabajo ofrecen una hoja de ruta clara para extender el estudio: combinando modelos de reconstrucción con filtros supervisados, trabajando con datos menos preprocesados y profundizando en la configuración de los VAE, se podría avanzar hacia sistemas de detección de fraude aún más precisos, robustos y adaptados a las necesidades operativas de entidades financieras reales.

Conclusión final.

Con los autoencoders como eje, se ha construido y evaluado un pipeline reproducible que mejora de forma consistente a un baseline lineal (PCA) en la detección de fraude, tanto en términos de separabilidad global (ROC-AUC) como de capacidad de captura de la clase positiva (sensibilidad), manteniendo al mismo tiempo una especificidad elevada y una precisión coherente con la prevalencia extremadamente baja del fraude.

Los resultados obtenidos muestran que, bajo las limitaciones del dataset disponible, los autoencoders especialmente el autoencoder estándar en el escenario sin balanceo constituyen una alternativa claramente superior a PCA para priorizar transacciones sospechosas en un contexto de fraude financiero. El uso del error de reconstrucción como score de anomalía y la definición de un umbral operativo proporcionan un mecanismo flexible para adaptar el sistema a distintos niveles de tolerancia al riesgo y a diferentes cargas de revisión.

Dado que el propósito del trabajo era aplicar la metodología de investigación y los contenidos del máster al desarrollo de modelos viables para la detección de transacciones fraudulentas, puede concluirse que este objetivo se ha cumplido. El estudio aporta un esquema metodológico completo datos, preprocesado, modelos, evaluación y lectura operativa y evidencia empírica suficiente para sostener que los autoencoders son una herramienta adecuada y prometedora en este ámbito, dentro del alcance definido para este Trabajo Final de Máster.

Referencias Bibliográficas

- [1] ULB Machine Learning Group. (2016). Credit card fraud detection [Data set]. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [2] Chollet, F., & others. (2015). Keras [Computer software]. <https://keras.io>
- [3] Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- [4] Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1–5.
- [5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
- [7] Jolliffe, I. T. (2002). Principal component analysis (2nd ed.). Springer.
- [8] Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. arXiv preprint arXiv:1312.6114.

Apéndice

A.1 Metodología

A.1.1 Preprocesado de Datos

```
1 # Convertimos el atributo de tiempo a horas del día
2 df['Time'] = df['Time'].apply(lambda t: (t/3600) % 24)
3
4 # Añadimos una cantidad insignificante para evitar tomar el logaritmo de 0
5 df['log10_amount'] = np.log10(df.Amount + 0.00001)
6
7 # Movemos el campo 'Class' al final del DataFrame
8 df = df[
9     [col for col in df if col not in ['Class', 'log10_amount']] +
10    ['log10_amount', 'Class']
11 ]
12
13 # Eliminamos la columna 'Amount'
14 df = df.drop(['Amount'], axis=1)
```

Figura A.1: Preprocesado de Datos

A.1.2 Partición sin Balanceo de Clases

```
1 # Mezclar el conjunto de entrenamiento
2 clean = clean.sample(frac=1).reset_index(drop=True)
3
4 # Dividir el conjunto limpio en entrenamiento y test
5 X_train, X_test_clean = train_test_split(clean, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
6
7 X_train, X_validate = train_test_split(X_train, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
8
9 # Dividir los datos de fraude para test y validate
10 fraud_test, fraud_validate = train_test_split(fraud, test_size=0.1, random_state=RANDOM_SEED) # El 10% de fraude va a validate
11
12 # Añadir transacciones fraudulentas al conjunto de test y mezclar
13 X_test = pd.concat([X_test_clean, fraud_test]).sample(frac=1).reset_index(drop=True)
14
15 # Añadir el 10% de las transacciones fraudulentas al conjunto de validate y mezclar
16 X_validate = pd.concat([X_validate, fraud_validate]).sample(frac=1).reset_index(drop=True)
```

Figura A.2: Partición sin Balanceo de Clases

A.1.3 Partición con Balanceo de Clases (SMOTE)

```
1 # Mezclar el conjunto de transacciones normales
2 clean = clean.sample(frac=1, random_state=RANDOM_SEED).reset_index(drop=True)
3
4 # Dividir el conjunto de transacciones normales en training, validación y test
5 X_train_v2, X_test_v2 = train_test_split(clean, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
6 X_train_v2, X_validate_v2 = train_test_split(X_train_v2, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
7
8 # Dividir los datos de fraude para train, validate, test
9 fraud_train_v2, fraud_test_v2 = train_test_split(fraud, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
10 fraud_train_v2, fraud_validate_v2 = train_test_split(fraud_train_v2, test_size=VALIDATE_SIZE, random_state=RANDOM_SEED)
11
12 # Combinar los datos para el conjunto de entrenamiento:
13 # Se unen los registros normales (X_train_v2) y los fraudes asignados a training.
14 X_train_combined_v2 = pd.concat([X_train_v2, fraud_train_v2]).sample(frac=1, random_state=RANDOM_SEED).reset_index(drop=True)
15 # Crear el vector de etiquetas: 0 para normales (correspondiente a X_train_v2) y 1 para fraudes (fraud_train_v2)
16 y_train_v2 = np.concatenate([np.zeros(len(X_train_v2)), np.ones(len(fraud_train_v2))])
17
18 # Verificar la distribución original en training (antes de balancear)
19 print("Antes de SMOTE, distribución en training:")
20 print(pd.Series(y_train_v2).value_counts())
21
22 # Para lograr una relación 70/30 (normales/fraudes) en training:
23 target_fraud_count = int((3/7) * len(X_train_v2))
24
25 # Definir la estrategia de SMOTE: dejar intacta la clase 0 (normales) y oversamplear la clase 1 (fraudes) hasta target_fraud_count
26 smote_strategy = {0: len(X_train_v2), 1: target_fraud_count}
27
28 sm = SMOTE(sampling_strategy=smote_strategy, random_state=RANDOM_SEED)
29 # ¡IMPORTANTE! Usar el conjunto combinado para que X e y tengan el mismo número de muestras.
30 X_train_bal, y_train_bal = sm.fit_resample(X_train_combined_v2, y_train_v2)
31
32 # Convertir el resultado a DataFrame y agregar la columna 'Class'
33 X_train_v2 = pd.DataFrame(X_train_bal, columns=X_train_combined_v2.columns)
34 X_train_v2['Class'] = y_train_bal
35
36 # Verificar la nueva distribución en training después de SMOTE
37 print("\nDespués de SMOTE, distribución en training:")
38 print(X_train_v2.Class.value_counts())
39
40 # Añadir transacciones fraudulentas al conjunto de test y mezclar
41 X_test_v2 = pd.concat([X_test_v2, fraud_test_v2]).sample(frac=1, random_state=RANDOM_SEED).reset_index(drop=True)
42
43 # Añadir transacciones fraudulentas al conjunto de validación y mezclar
44 X_validate_v2 = pd.concat([X_validate_v2, fraud_validate_v2]).sample(frac=1, random_state=RANDOM_SEED).reset_index(drop=True)
```

Figura A.3: Partición con Balanceo de Clases (SMOTE)

A.1.4 Preparación de las Características y Etiquetas

```
1 # Obtener las etiquetas de clase para entrenamiento, validación y test
2 y_train, y_validate, y_test = X_train.Class.values, X_validate.Class.values, X_test.Class.values
3
4 # Eliminar la columna 'Class' de los conjuntos de entrenamiento, validación y test
5 X_train = X_train.drop('Class', axis=1)
6 X_validate = X_validate.drop('Class', axis=1)
7 X_test = X_test.drop('Class', axis=1)
8
9 # Obtener las etiquetas de clase para entrenamiento, validación y test
10 y_train_v2, y_validate_v2, y_test_v2 = X_train_v2.Class.values, X_validate_v2.Class.values, X_test_v2.Class.values
11
12 # Eliminar la columna 'Class' de los conjuntos de entrenamiento, validación y test
13 X_train_v2 = X_train_v2.drop('Class', axis=1)
14 X_validate_v2 = X_validate_v2.drop('Class', axis=1)
15 X_test_v2 = X_test_v2.drop('Class', axis=1)
```

Figura A.4: Preparación de las Características y Etiquetas

A.1.5 Normalización de Datos

```
1 # Creamos una instancia del escalador MinMaxScaler
2 min_max_scaler = MinMaxScaler()
3
4 # Ajustamos el escalador a los datos de entrenamiento
5 min_max_scaler.fit(X_train)
```

MinMaxScaler ⓘ ?

MinMaxScaler()

```
1 # Creamos una instancia del escalador MinMaxScaler
2 min_max_scaler_v2 = MinMaxScaler()
3
4 # Ajustamos el escalador a los datos de entrenamiento
5 min_max_scaler_v2.fit(X_train_v2)
```

Figura A.5: Preparación de las Características y Etiquetas

```
1 # Utilizamos los parámetros aprendidos por el escalador MinMaxScaler para normalizar los conjuntos de entrenamiento y validación.
2 X_train_transformed = min_max_scaler.transform(X_train)
3 X_validate_transformed = min_max_scaler.transform(X_validate)
4
5 # Transformamos el conjunto de prueba
6 X_test_transformed = min_max_scaler.transform(X_test)
```

```
1 # Utilizamos los parámetros aprendidos por el escalador MinMaxScaler para normalizar los conjuntos de entrenamiento y validación.
2 X_train_transformed_v2 = min_max_scaler_v2.transform(X_train_v2)
3 X_validate_transformed_v2 = min_max_scaler_v2.transform(X_validate_v2)
4
5 # Transformamos el conjunto de prueba
6 X_test_transformed_v2 = min_max_scaler_v2.transform(X_test_v2)
```

Figura A.6: Preparación de las Características y Etiquetas

A.1.6 Función de Curvas del Entrenamiento

```
1 def plot_training_history_saves(hist):
2     # Configura el tamaño de la figura para acomodar los gráficos superiores e inferiores
3     plt.figure(figsize=(12, 10))
4
5     # Gráfico superior para la evolución del loss
6     plt.subplot(2, 2, 1) # 2 filas, 2 columnas, posición 1
7     plt.plot(hist['loss'], label="loss")
8     plt.plot(hist['val_loss'], label="val_loss")
9     plt.title("Evolución del Loss Durante el Entrenamiento")
10    plt.xlabel("Épocas")
11    plt.ylabel("Loss")
12    plt.legend()
13
14    # Gráfico superior para la evolución de la métrica de precisión
15    plt.subplot(2, 2, 2) # 2 filas, 2 columnas, posición 2
16    plt.plot(hist['acc'], label="acc")
17    plt.plot(hist['val_acc'], label="val_acc")
18    plt.title("Evolución de la Precisión Durante el Entrenamiento")
19    plt.xlabel("Épocas")
20    plt.ylabel("Precisión")
21    plt.legend()
22
23    # Selección de las últimas 100 épocas para los gráficos inferiores
24    epochs_shown = 100
25    epochs = range(len(hist['loss']))
26    last_epochs = list(epochs)[-epochs_shown:]
27
28    # Gráfico inferior para la evolución del loss en las últimas 100 épocas
29    plt.subplot(2, 2, 3) # 2 filas, 2 columnas, posición 3
30    plt.plot(last_epochs, hist['loss'][-epochs_shown:], label="loss")
31    plt.plot(last_epochs, hist['val_loss'][-epochs_shown:], label="val_loss")
32    plt.title("Loss en las Últimas 100 Épocas")
33    plt.xlabel("Últimas Épocas")
34    plt.ylabel("Loss")
35    plt.legend()
36
37    # Gráfico inferior para la evolución de la métrica de precisión en las últimas 100 épocas
38    plt.subplot(2, 2, 4) # 2 filas, 2 columnas, posición 4
39    plt.plot(last_epochs, hist['acc'][-epochs_shown:], label="acc")
40    plt.plot(last_epochs, hist['val_acc'][-epochs_shown:], label="val_acc")
41    plt.title("Precisión en las Últimas 100 Épocas")
42    plt.xlabel("Últimas Épocas")
43    plt.ylabel("Precisión")
44    plt.legend()
45
46    # Mostrar los gráficos
47    plt.tight_layout()
48    plt.show()
49
```

Figura A.7: Función de Curvas del Entrenamiento

A.1.7 Función Calcular Umbral

```
1 def calcular_umbral(autoencoder):
2     # Obtener reconstrucciones para el conjunto de entrenamiento
3     reconstrucciones_train = autoencoder.predict(X_train_transformed)
4
5     # Calcular pérdida de reconstrucción MSE para el conjunto de entrenamiento
6     mse_train = np.mean(np.power(X_train_transformed - reconstrucciones_train, 2), axis=1)
7
8     # Calcular el umbral como el percentil 95 del MSE del conjunto de entrenamiento
9     umbral = np.percentile(mse_train, 95)
10
11     return umbral
```

Figura A.8: Función Calcular Umbral

A.1.8 Función Evaluar Rendimiento Autoencoder

```
1 def evaluar_rendimiento_autoencoder(autoencoder):
2     # Calcular el umbral como el percentil 95 del MSE
3     umbral = calcular_umbral(autoencoder)
4     print(f"Umbral para la detección de outliers: {umbral} \n")
5
6     # Obtener reconstrucciones para el conjunto de validación
7     reconstrucciones_validate = autoencoder.predict(X_validate_transformed)
8
9     # Calcular pérdida de reconstrucción MSE para el conjunto de validación
10    mse_validate = np.mean(np.power(X_validate_transformed - reconstrucciones_validate, 2), axis=1)
11
12    # Presentar el error de reconstrucción MSE medio del conjunto de validación
13    print(f"Error de reconstrucción MSE medio en el conjunto de validación: {np.mean(mse_validate)}\n")
14
15    # Separar MSE por clase en el conjunto de validación
16    limpio = mse_validate[y_validate == 0]
17    fraude = mse_validate[y_validate == 1]
18
19    # Graficar distribución
20    plt.hist(limpio, bins=50, density=True, label="Limpio", alpha=.6, color="green")
21    plt.hist(fraude, bins=50, density=True, label="Fraude", alpha=.6, color="red")
22    plt.axvline(x=umbral, color='r', linestyle='--', label='Umbral de Outlier')
23    plt.title("(Normalizada) Distribución de la Pérdida de Reconstrucción")
24    plt.legend()
25    plt.show()
26
27    # Clasificar como outlier basado en umbral en el conjunto de validación
28    predicciones = (mse_validate > umbral).astype(int)
29
30    # Matriz de confusión
31    cm = confusion_matrix(y_validate, predicciones)
32
33    LABELS = ["Normal", "Fraud"]
34    plt.figure(figsize=(8, 8))
35    sns.heatmap(cm, xticklabels=LABELS, yticklabels=LABELS, annot=True, fmt="d");
36    plt.title("Confusion matrix")
37    plt.ylabel('True class')
38    plt.xlabel('Predicted class')
39    plt.show()
40
41    tn, fp, fn, tp = cm.flatten()
42
43    print(f"""
44    % de transacciones etiquetadas como fraude que fueron correctas (precisión): {tp}/{(fp+tp)} = {tp/(fp+tp):.2%}
45    % de transacciones fraudulentas que fueron detectadas con éxito (sensibilidad): {tp}/{(fn+tp)} = {tp/(fn+tp):.2%}
46    % de transacciones normales que fueron identificadas correctamente (especificidad): {tn}/{(tn+fp)} = {tn/(tn+fp):.2%}""")
```

Figura A.9: Función Calcular Umbral

A.2 Modelos

A.2.1 Autoencoder

```
autoencoder_v3_v34 = Sequential([
    # Encode
    Dense(input_dim, activation='elu', input_shape=(input_dim,)),
    BatchNormalization(),
    Dropout(0.1),
    Dense(32, activation='elu'),
    Dense(16, activation='elu'),
    BatchNormalization(),
    Dropout(0.1),
    Dense(8, activation='elu'),

    Dense(4, activation='elu', activity_regularizer=l1(0.01)), # Añadido una capa adicional

    # Decode
    Dense(8, activation='elu'),
    Dense(16, activation='elu'),
    Dense(32, activation='elu'),
    BatchNormalization(),
    Dropout(0.1),
    Dense(input_dim, activation='sigmoid'),
])

# Configuración del optimizador Adam con el schedule de tasa de aprendizaje
optimizer = Adam(learning_rate=0.000005)

autoencoder_v3_v34.compile(optimizer=optimizer,
                             loss="mse",
                             metrics=["acc"])

autoencoder_v3_v34.summary()
```

Figura A.10:Ejemplo Autoencoder

A.2.2 Autoencoder Variacional

```
1 def build_new_vae_model_7():
2     # Encoder con activación 'elu'
3     inputs = Input(shape=(input_dim,), name='encoder_input')
4     x = Dense(units=input_dim, activation='elu', kernel_regularizer=l1(0.001))(inputs)
5     x = BatchNormalization()(x)
6     x = Dropout(0.1)(x)
7     x = Dense(units=64, activation='elu')(x)
8     x = BatchNormalization()(x)
9     x = Dense(units=32, activation='elu')(x)
10    x = BatchNormalization()(x)
11
12    # Cuello de botella con latent_dim=4
13    latent_dim = 4
14    z_mean = Dense(latent_dim, name='z_mean')(x)
15    z_log_var = Dense(latent_dim, name='z_log_var')(x)
16    z = Lambda(sampling, name='z')([z_mean, z_log_var])
17
18    # Decoder con activación 'relu'
19    decoder_input = Input(shape=(latent_dim,), name='z_sampling')
20    x_dec = Dense(units=32, activation='relu')(decoder_input)
21    x_dec = BatchNormalization()(x_dec)
22    x_dec = Dense(units=64, activation='relu')(x_dec)
23    x_dec = BatchNormalization()(x_dec)
24    outputs_dec = Dense(input_dim, activation='sigmoid')(x_dec)
25    decoder = Model(decoder_input, outputs_dec, name='decoder')
26
27    outputs = decoder(z)
28    outputs_with_loss = VAELossLayer(name='vae_loss')([inputs, outputs, z_mean, z_log_var])
29
30    vae = Model(inputs, outputs_with_loss, name='new_vae_model_7')
31    optimizer = Adam(learning_rate=1e-5)
32    vae.compile(optimizer=optimizer,
33               loss="mse",
34               metrics=["acc", "mse"])
35    return vae
36
37 model_variant2_7 = build_new_vae_model_7()
```

Figura A.11: Ejemplo Autoencoder Variacional

A.2.3 Función Sampling VAE

```
1 # Función de muestreo (reparametrization trick)
2 def sampling(args):
3     z_mean, z_log_var = args
4     batch = K.shape(z_mean)[0]
5     dim = K.shape(z_mean)[1]
6     epsilon = K.random_normal(shape=(batch, dim))
7     return z_mean + K.exp(0.5 * z_log_var) * epsilon
```

Figura A.12: Función Sampling

A.2.4 Función VAE Loss Layer VAE

```
1 # Capa personalizada que agrega la pérdida VAE (reconstrucción + KL)
2 input_dim = X_train_transformed_v2.shape[1]
3
4 class VAE Loss Layer(tf.keras.layers.Layer):
5     def __init__(self, **kwargs):
6         super(VAE Loss Layer, self).__init__(**kwargs)
7     def call(self, inputs):
8         original, reconstruction, z_mean, z_log_var = inputs
9         reconstruction_loss = tf.reduce_mean(tf.square(original - reconstruction)) * input_dim
10        kl_loss = -0.5 * tf.reduce_mean(tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1))
11        total_loss = reconstruction_loss + kl_loss
12        self.add_loss(total_loss)
13        return reconstruction
```

Figura A.13: Función VAE Loss Layer